

Astana IT University

**SUNGAT KAPAROV
ALEMKHAN YERGALIEV
SULTANBEK KHASSEN**

**Development of Cryptocurrency
transferring service**

6B06102 — Educational Program

Diploma project

Supervisor
Khaimuldin A.
MSc in Engineering
Senior Lecturer

Kazakhstan Republic
Nur-Sultan, 2022

CONTENTS

Annotation	3
Definitions	4
Introduction	5
1 Theoretical part	7
1.1 Literature review	7
1.2 Secondary Research and Analysis	7
1.3 Alternatives comparison	13
1.4 Technology stack	14
2 Practical part	18
2.1 Research methodology	18
2.2 Client-Server interaction	19
2.3 Software architecture	19
2.4 Services description	21
2.4.1 The Authentication Service	21
2.4.2 The Lot Service	21
2.4.3 The Wallet Service	23
2.4.4 The Crypto Service	27
2.4.5 The Trade Service	30
2.4.6 The Notification Service	33
2.4.7 Appeal service	36
2.4.8 Chat service	39
2.4.9 API Gateway	41
Conclusion	43
2.5 Further research	44
Bibliography	45

Annotation

Relevance: It is important to develop a local cryptocurrency transferring service due to its effect on economies and people's involvement in this sphere in order to prevent leading and dependency of abroad solutions.

The Project goal: To develop a cryptocurrency transferring service according to drawbacks of existing solutions and their best practices in order to use the best methods that are presented nowadays with our additional improvements and algorithms. To achieve the project goal there are several objectives that were formulated.

Objectives: Find out the current ways of crypto transferring Define the main problems and its possible solutions of the current transferring methods Evaluate the best practices and technologies related to crypto transfer theme Develop a trustful and easily accessible system for cryptocurrency transferring

According to our secondary research we found out different existing services that are related to cryptocurrency transferring such as Binance, Paxful, Blockchain.com. After concluding the results of the secondary research we decided to develop our service similarly as Binance, but with our own solutions in technology stack, architecture and business logics with algorithms. Results and Conclusion: Summarizing the relevance, goal, objectives and research that has been done, we developed a brand new service that offers: Reliable service that offers P2P transfers for crypto-users(firstly in ERC20, Ethereum; further in parachains). Kazakhstani blockchain related product. Future possibility to include P2P transfers of different cryptocurrency types between one another.

Definitions

Following terms are used in this work:

Cryptocurrency	is any form of currency that exists digitally or virtually and uses cryptography to secure transactions.
Peer-to-peer(P2P) payments	are money transfers from one person to another.
Blockchain	is a distributed database that is shared among the nodes (other machines) of a computer network.
Smart contracts	are simply programs stored on a blockchain that run when predetermined conditions are met (rules).
Tokens	are virtual currency tokens or a denomination of a cryptocurrency. It represents a tradable asset or utility that resides on its own blockchain and allows the holder to use it for investment or economic purposes.
ERC-20	has emerged as the technical standard; it is used for all smart contracts on the Ethereum blockchain for token implementation and provides a list of rules that all Ethereum-based tokens must follow.
Keystore	JSON-encoded file that contains a single (randomly generated) private key, encrypted by a passphrase for extra security. Keystores are a standard way to store private keys locally to let clients such as Geth handle privateKey/sign in for you. Nethereum offers a dedicated 'keystore' service to facilitate the creation and management of keystore files.

INTRODUCTION

From ancient times society gave some value to every object in order to use them for equivalent exchange. After a long period humanity created a new model for exchanging objects – valued money currencies. And money became one of the most significant resources along with others such as food, energy, or health. By this moment, there are a lot of types of money currencies, and one of them are cryptocurrencies.

In the digital world people started investing into cryptocurrencies due to its high values. Cryptocurrency - digital type of money that turned into the form of tokens or “coins” where transactions are confirmed and records maintained by a decentralized system using cryptography, rather than by a centralized authority. For example: Bitcoin, Ethereum, Litecoin and others (Hayes, 2021). The world’s first cryptocurrency appeared in 2009, and it is Bitcoin, since 2009 over 1000 tradable tokens have been born. The majority of people believe that there is no doubt that digital currencies will be the main thing to invest in the near future. Therefore, investors need an easy and fast platform for selling and buying cryptocurrencies. Currently, digital money transfers work with a trading system. A cryptocurrency exchange system is a platform that works as a trading system with other types of coins. It is also known as a digital currency exchange (DCE). Nowadays, cryptocurrency exchanges are real by marketplaces such as Koinex, Binance, Zebpay (Amit Bauriya, 2019). Despite current ways of transferring, people want to sell and buy cryptocurrencies directly using typical known currencies such as USD, EUR, or TG, and they are looking for ebay, amazon styled marketplace. In addition, it is important to consider flows of coins in the transactional ledger and tracking as coins move across their ledgers (Haaron Yousaf, 2019). Therefore, there is a need to implement a new platform which overcomes previous systems weaknesses.

In order to develop our project, the next goal has been formulated: To develop a cryptocurrency transferring service according to drawbacks of existing solutions and their best practices in order to use the best methods that are presented nowadays with our additional improvements and algorithms. In addition to cover our project goal the following objectives has been created:

Objectives:

- 1 Find out the current ways of crypto transferring
- 2 Define the main problems and its possible solutions of the current transferring methods
- 3 Evaluate the best practices and technologies related to crypto transfer theme

4 Develop a trustful and easily accessible system for cryptocurrency transferring

The paper has been divided into 5-6 sections. The first part is the introduction. The second part includes a theoretical information with literature review where all materials related to our topic will be reviewed and that will help to reach my objectives. The fourth part of this paper will be related to practical data and contains methodology discussions of the study where qualitative and quantitative analysis will be presented in order to compare them with other research results. The last parts will cover the conclusion where our project work will be summed up.

1 Theoretical part

1.1 Literature review

In the last decade, especially the near past, the cryptocurrency market has been developed with a high speed. Knowledge in the cryptocurrencies and its market will pay off very quickly if the development of the electronic money industry remains the same. Therefore, there is a high priority to investigate a lot about the crypt industry, how they work, what currencies are more valuable, how to sell and buy them and a plethora of other questions that need to be answered. This literature review will cover the main aspects of such questions and they will be comprehensively analyzed in order to provide detailed information. Particular attention will be given to crypto marketplaces.

The second section of this paper represents an overview of the cryptocurrencies including its structure, used technologies, important notes, and famous digital currencies such as Bitcoin and Ethereum. In addition, it provides in-depth analysis of cryptocurrency markets, the ways of exchanging the currencies, and analysis of trading risks, its reasons, and solutions. Furthermore, the second section gives some proof of the new cryptocurrency transferring system's necessity.

1.2 Secondary Research and Analysis

The ideas related to digital currencies appeared in the late 1980s, however electronic money implementations first started appearing after 2009s when the most famous and the first decentralized cryptocurrency as Bitcoin was born by a pseudonymous developer named Satoshi Nakamoto [1]. Furthermore, the term cryptocurrency has revealed itself to the public usage after Nakamoto created Bitcoin, and it was called as a peer-to-peer currency first. According to Brett Scott, journalist who have written the book "The Heretic's Guide to Global Finance: Hacking the Future of Money", cryptocurrency can be understood as a "system intended for the issuance of tokens which are intended to be used as a general or limited-purpose medium-of-exchange, and which are accounted for using an often collectively-maintained digital ledger making use of cryptography to replace trust in institutions to varying extents. Against such a backdrop, the singular term cryptocurrency can mean a token, intended to be used as a general or limited-purpose medium-of-exchange, issued via a cryptocurrency system." [2]. Also, one of the safest cryptocurrency exchange platforms named Coinbase provided a less detailed but briefly written and understandable definition as: Cryptocurrency is decentralized virtual money that can be used over the internet. The word decentralized is important while defining the cryptocurrency, because it means that if anyone owns any digital money, it will be managed only by his owner, and there are no third-party traders as banks or brokers [3]. Most cryptocurrencies are

built using blockchain technology. And this technology also became widely known when the Bitcoin network first launched. In short, Blockchains are a tamper-proof digital ledger implemented without a central authority. With the help of blockchain technology users are able to record transactions on shared blocks or ledgers within that community, and those recorded transactions will be blocked for changing once published. And instead of centralized authority blockchain uses cryptographic mechanisms to protect digital cashes.

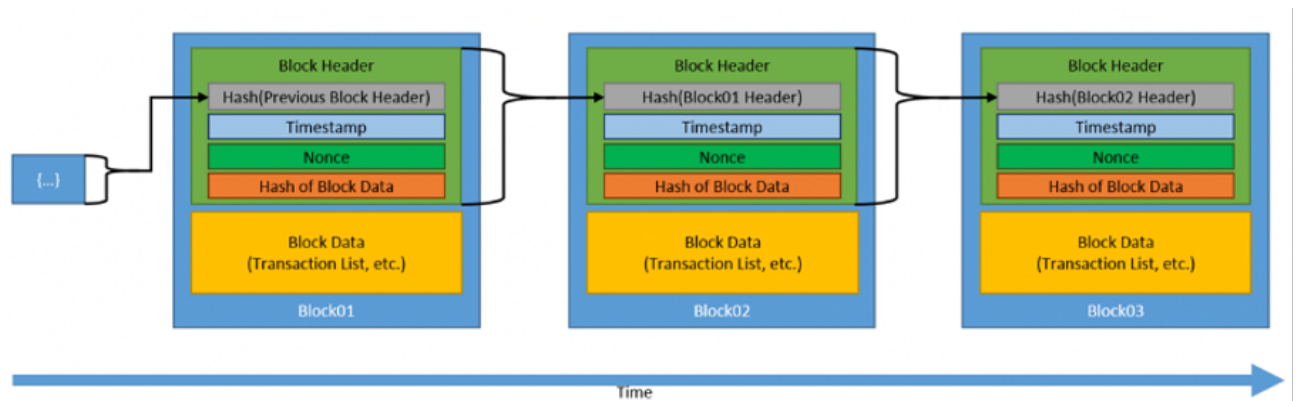


Figure 1.1 – Generic Chain of Blocks

Figure 1.1 illustrates how blockchains are organized, and this structure explains why there is no need for a trusted third-party broker; each user in the system can verify the system's integrity [4]. Nowadays there are a lot of cryptocurrencies on the market, but the most valuable and famous is Bitcoin. The graph below proves that cryptocurrency market capitalization has increased mostly thanks to Bitcoin (BTC).

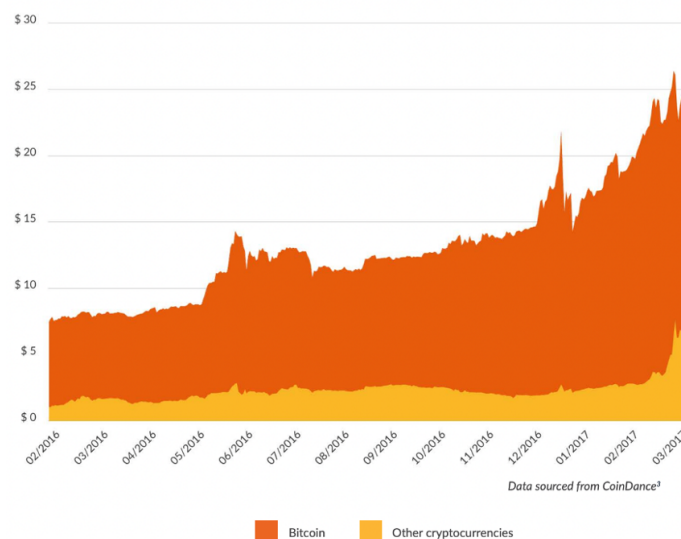


Figure 1.2 – Cryptocurrency market capitalization from 2016 to 2017

The second most famous cryptocurrency is called Ethereum (ETH). Ethereum is also a decentralized currency which first launched in 2015 and works using Turing-complete programming language with the system of scripts and contracts executed by every participating node [5].

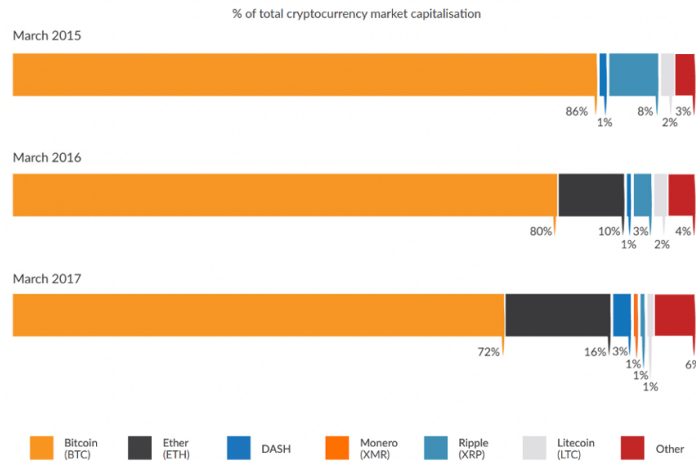


Figure 1.3 – Market capitalization share between different cryptocurrencies

The 3rd figure demonstrates the capitalization division between several cryptocurrencies from 2015 to 2017, where Bitcoin is the absolute leader although it dropped from 86 percent to 72 percent, while the second largest currency the Ethereum network increased for 6 percent in one year.

	Bitcoin	Ethereum	DASH	Ripple	Monero	Litecoin
Q1 2016	201,595	20,242	1,582	N/A	579	4,453
Q2 2016	221,018	40,895	1,184	N/A	435	5,520
Q3 2016	219,624	45,109	1,549	N/A	1,045	3,432
Q4 2016	261,710	42,908	1,238	N/A	1,598	3,455
January - February 2017	286,419	47,792	1,800	N/A	2,611	3,244

Figure 1.4 – Average daily number of transactions for each the most famous cryptocurrencies

According to the table illustrated in the 4th figure, each currency's number of transactions increased except Litecoin. The major development can be noticed on Ethereum and Bitcoin currencies. Despite Bitcoin showing the highest number

of transactions, the Ethereum network increased more than twice from 2016. If we make a short summary after analyzing the above figures, there is a good development trend that can be noticed related to the Ethereum network [5].

The cryptocurrency market is developing year by year actively. After Bitcoin entered our world, about 1500 other types of cryptocurrencies appeared, and 600 out of them are actively trading today. Trading of these digital money happens with the goal of media of exchange payments, especially for speculation purposes. So, in the beginning of 2017s the cryptocurrency market capitalization reached 91 billion dollars. Furthermore, to rate the market stability there were provided some graphs that shows the difference between existing currencies and active currencies, and the lines that illustrates the birth and death rates of digital assets.

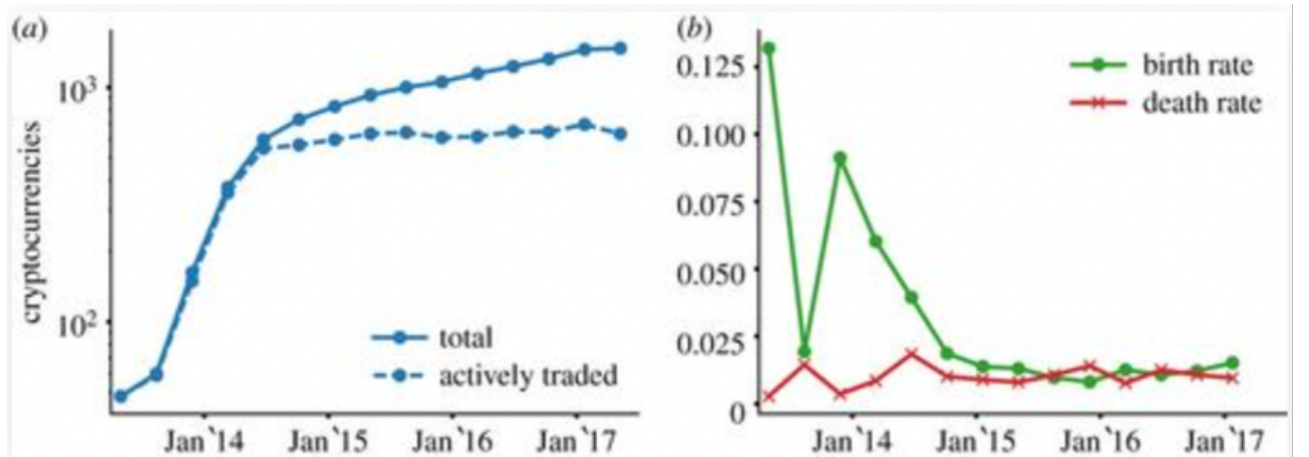


Figure 1.5

The 5th figure's (a) part demonstrates that the difference between total number of digital currencies and actively traded currencies is not too high. It proves the validity and perspective of the cryptocurrency market, and there are big opportunities of getting good income if there is another good trading platform. The (b) part provides information about cryptocurrencies' birth and death rate, and due to the fact that birth rate and death rate remains same and equal since 2014, the actively traded digital currencies is stable, the market is stable [6].

There are different methods of trading and exchanging cryptocurrencies. It is important to consider and know several types of exchanges because they can be traded to other digital assets or national currencies, also there exist other things that have a significant role for trading processes such as liquidity or price discovery. This project's main goal is to implement a cryptocurrency transferring service with the maximum freedom of price and commission, which will allow

users to trade their digital money using simple national currencies.

Type of activity	Description
Order-book exchange	Platform that uses a trading engine to match buy and sell orders from users
Brokerage service	Service that lets users conveniently acquire and/or sell cryptocurrencies at a given price
Trading platform	Platform that provides a single interface for connecting to several other exchanges and/or offers leveraged trading and cryptocurrency derivatives

Figure 1.6 – Types of exchanges

The 6th figure demonstrates a table with the basic methods of cryptocurrency exchanges. The Brokerage service with some features of Trading platform fits well to the purposes of this project.

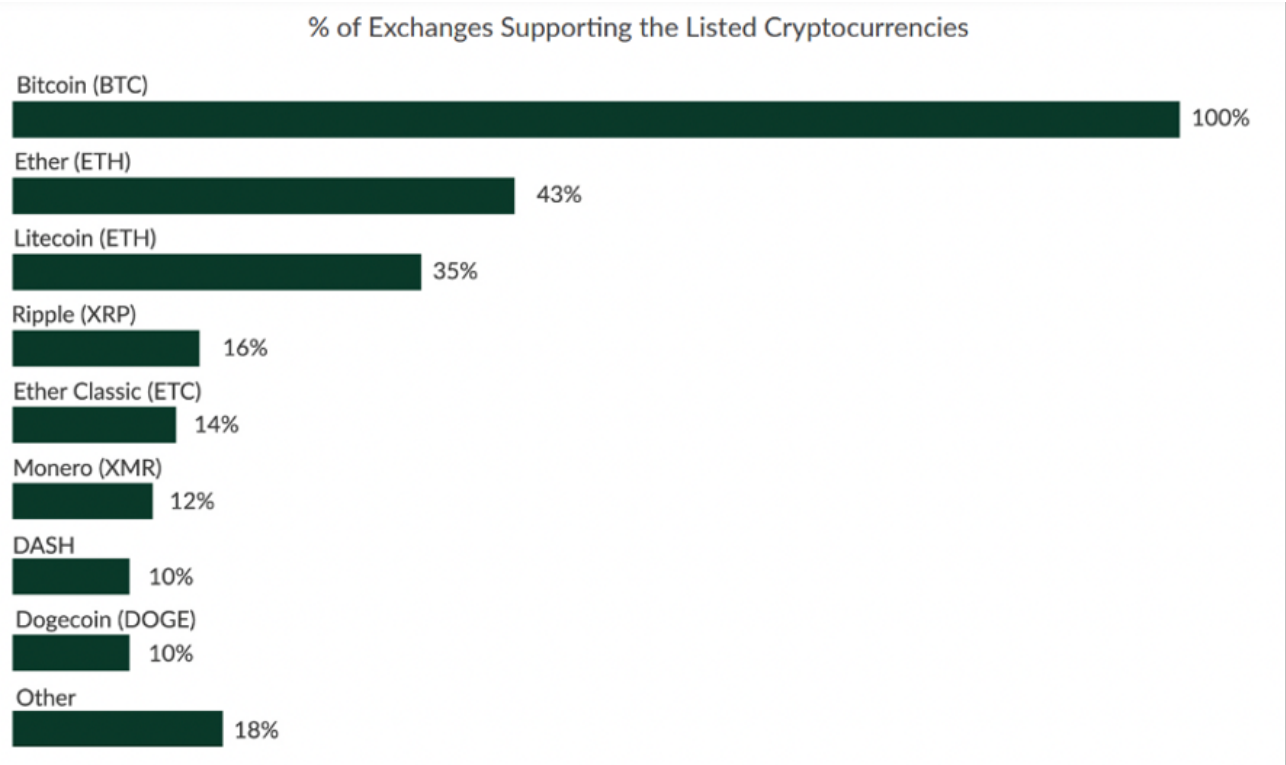


Figure 1.7 – Cryptocurrency percentage which supports exchanges

According to the data illustrated on the figure 7, although there exist a lot of different marketplaces, some of the currencies still need a platform for exchanging. Approximately 40 percent of small trading goes with the help of Brokerage services, and such small exchanges have much more risks than large

exchanges. The majority of risks are related to security breaches which result in a loss of funds.

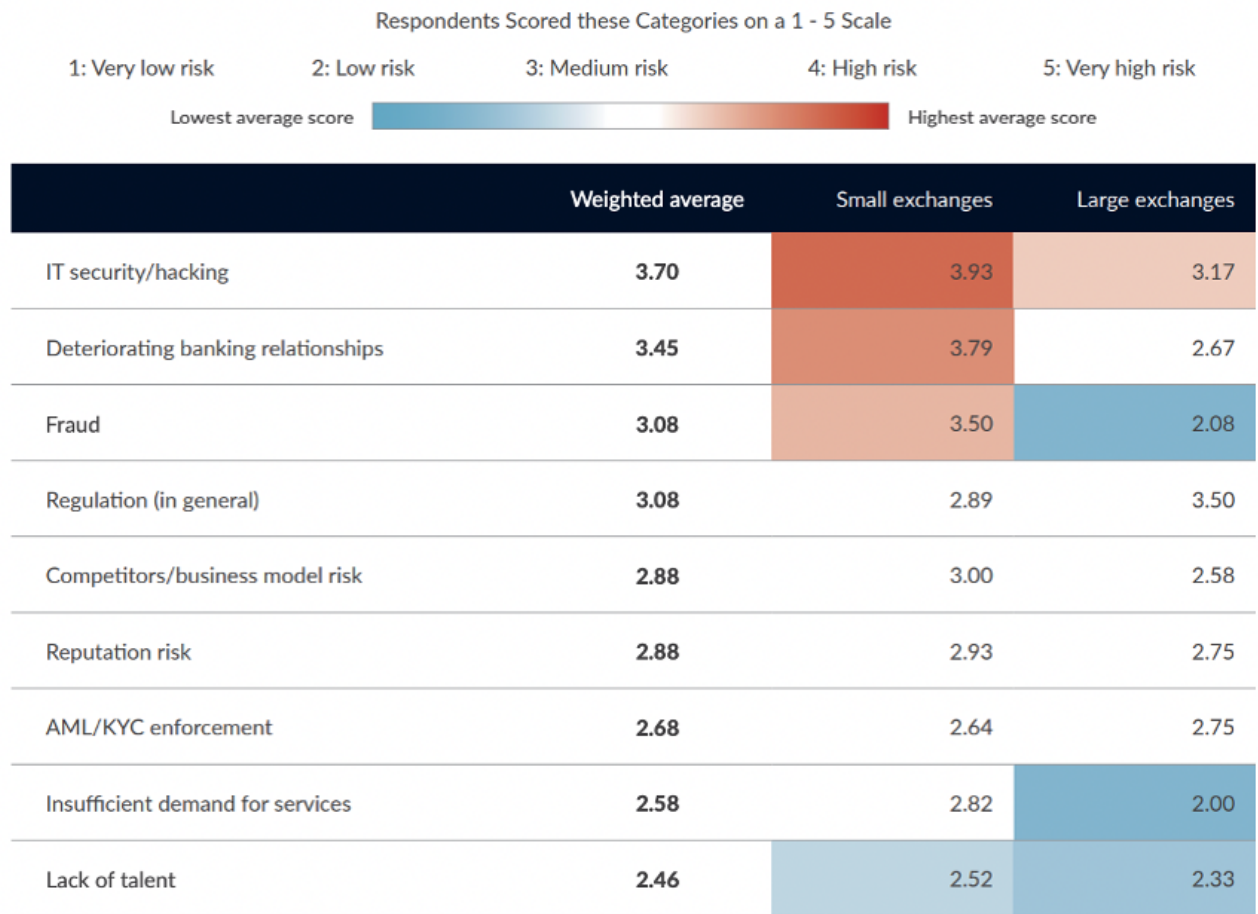


Figure 1.8 – Cryptocurrency exchanging operational risks in numbers

Considering survey results represented on the 9th figure, little typed exchanges have challenges with deteriorating banking relationships, also they are not fault tolerant related to hacking (Rauchs, 2017). Therefore, the new cryptocurrency transferring service should overcome these kinds of drawbacks. The possible solutions are creating two factor authentication, confirming a cryptocurrency exchanging transaction using two end user's confirmation, and making a centralized debating system to handle disputing situations.

In conclusion to these sections, cryptocurrencies are really occupying the digital investments with their technological advantages, so by the end of 2017 their market capitalization has attained 91 billion dollars. Most of the market capitalization belongs to Bitcoin and Ethereum. Additionally, the market capitalization is increasing by years and the actuality of the digital assets remains the same due to its equal birth and death rates. Another important thing is that

the research proved that the system of exchanging currencies is not ideal, and it can not be. However, the reasons for the latest disadvantages of the exchanging platforms have been revealed. To sum up these insights, the new cryptocurrency transferring system have to:

- Find out the current ways of crypto transferring
- Define the main problems and its possible solutions of the current transferring methods
- Evaluate the best practices and technologies related to crypto transfer theme
- Develop a trustful and easily accessible system for cryptocurrency transferring

1.3 Alternatives comparison

There are such alternatives to our project as blockchain.com, binance and paxful.

Table 1.1 – Alternatives

Name	Number of supported crypto	Software functions	Base price	Service fees	Payment	Pros
Binance	200+	Deposit, Trade, Withdraw	0 USD	0.1 %	credit and debit card, paypal	Multiple crypto support, high security, built-in exchange
Blockchain	7	Buy, Sell, Trade	0 USD	0.4 %	credit and debit card	Low transaction fees, built-in trading functions
Paxful	3	Buy, Sell, Trade	N/A	0.1-5 %	credit and debit card, paypal	P2P trades

Cons

Blockchain: Low anonymity, centralized, private keys stored in their servers

Binance: Difficult for beginners, mediocre customer support

Paxful: Few crypto support, service withdraw fees

According to the results related to alternative platforms, we decided to make our marketplace as binance, but with more simplicity. There will not be any fees, complicated features and a lot of crypto currencies at the beginning. However, the project will be easily scalable due to its architecture. There will be supported ERC20 and ETH trading for p2p.

1.4 Technology stack

The project provides cryptocurrency transferring service as a web application. Usually, web applications are based on backend and frontend parts, however there are several important aspects in developing any software, such as software architecture patterns, technology stack and methods of interaction. Without these aspects that should be clearly organized and well-defined, any project can be turned into legacy code after years of production. It will cause a lot of different issues related to scalability and development of a product in the future. Such applications that are not architected in a correct way in the beginning of its development are basically difficult to change, and they have a lot of dependencies that can ruin further development.

There are top 5 commonly used and basic software architecture patterns according to Mark Richards such as Layered architecture, Event-Driven architecture, Microkernel architecture, Microservices architecture pattern and Space-Based architecture.

Layered architecture that is known as n-tier architecture pattern is the most popular technique because of the Model View Controller (MVC) approach. The advantages of layered architecture are easy to maintain and test. It is the best option for applications that need to be developed quickly in a Monolithic way.

Event-driven architecture is designed to build applications that usually waits for something to happen. For example, the work of services of this architecture depends on the client side. And it has a central service that divides client requests into separate modules. Microkernel architecture is mostly used with a core set of operations in applications.

Microservices architecture as well as event-driven architecture and microkernel architecture is designed to handle scalability issues and efficiently separate client requests. The key point of microservices architecture approach is running independent services. It is suitable for web applications with small components and rapidly growing softwares [7].

Overall, the most suitable software architecture pattern for our project seems to be Microservices architecture due to its leading advantages and patterns compared to others. Because, cryptocurrency transferring service is a web application that consists of small modules, and it is planned to scale up for trading of other crypto coins. The project has authentication, lot, wallet, crypto, trade, appeal and notification services that are managed by the api gateway module.

Each service described above is an independent little application implemented using different technologies.

Authentication module:

- **Python:** Object-oriented, interpreted and high level programming language that have a lot of use cases.
- **FastApi:** High performance web framework that can be used to build APIs.
- Postgresql
- REST
- Docker
- **JWT:** Open-standard that is commonly used for authentication, it is an encoded string that contains data in json format

Lot module:

- Python
- **Django:** Open-source web framework designed for python that allows developers to build web applications faster and clean.
- Postgresql
- REST
- Docker

Wallet and Crypto module:

- **C#:** Object-oriented programming language developed by Microsoft. In this project, C# is used in Wallet, Crypto, Appeal and Chat services.
- **MongoDB:** Non-relational NoSQL database for storing data in documents format. Documents are stored in BSON format which is similar to JSON standard. This database is used in crypto and wallet services, and the choice of that will be explained in the description of these services. [8]

- **ASP.NET Core 6.0:** Open-source cross platform web framework for developing web applications using .NET platform. Version is latest. [9]

- **.NET 6.0:** Platform for developing software using .NET compatible programming languages such as C#, F#, Visual Basic.

- **Nethereum:** .NET library developed for working with Ethereum by interacting with Ethereum blockchain nodes such as Ganache, Geth, Parity. [10]

- **Ganache:** Local blockchain which allows users to interact with Ethereum by providing test accounts and deploying transactions and smart contracts with their further function calls.

- **Solidity:** Programming language for developing smart contracts running on Ethereum platform. [11]

- **Docker:** Software for automated deployment of applications running in virtual containers.

Appeal module:

- C#

- **Entity Framework Core:** Object relational mapping technology of access to outer data storages. Developed by Microsoft and fully works with .NET platform [12]

- ASP.NET Core 6.0

- **MailKit:** .NET library with support of sending IMAP, POP3 messages (email).

- **MySQL:** Open source relational database management system, developed by Oracle.

- Docker

Chat module:

- C#

- **SignalR:** .NET library, developed by Microsoft, which allows real-time communication by sending asynchronous messages between server and client.

- **Redis:** Open source NoSQL database which supports 'key-value' data storage.

Trade module:

- Python

- FastApi
- Postgresql
- REST
- Docker
- **Celery**: Open-source task queue that manages message passing by scheduling and working asynchronously.

Notification module:

- Python
- FastApi
- Postgresql
- REST
- Docker
- **Redis**: A key-value database that can operate as a message broker or cache.

API Gateway:

- C#
- ASP.NET Core
- **Ocelot**:

Ocelot is a .NET API Gateway open-source project communicating using REST. It is used in microservices oriented projects which require a unique point of requests from client to other microservices.

- Authentication.JWTBearer Official package from Microsoft which adds support with JWT authentication.

2 Practical part

2.1 Research methodology

This project paper's main aim is to find out the problems related to cryptocurrencies and its flow to implement a new platform that will cover all revealed issues and present them.

The project work should be well organized with a lot of information about my topic to cover objectives described above, and to do this we used secondary research methods to gather information. The most helpful method to collect data is secondary research because there are a lot of expert opinions, analysis, and ideas from different articles in some websites.

We have done my secondary research using different articles which we found using google scholar. Google scholar can be considered as a credible tool for searching information, because it can find a lot of useful resources. Those articles have some statistics of cryptocurrency market capitalization, their market validation and other important key points. To find trustful and reliable sources, I analyzed each article with the criteria of credibility with RAVEN methodology where RAVEN stands for:

R – Reputation (Does the source's history or status suggest reliability or unreliability?)

A – Ability to See (Is the source in a position to know what they're talking about. Can their observations be trusted?)

V – Vested Interest (Has the source of information anything personally at stake?)

E – Expertise (Does the source have specialized knowledge and does the situation demand it?)

N – Neutrality (Is the source predisposed to support a particular point of view for reasons other than vested interest?)

Sources that were used in literature review are credible, so the reputation of some works were good, and there was an expertise. For instance: Dr. Garrick Hileman and Michel Rauch's cooperated work called "Global cryptocurrency benchmarking study" can be represented as the most reliable and helpful resource, because their work provided many statistical information illustrated on the graphs and tables which can be considered as a quantitative data, also their expertise can be proved by The Cambridge Center for Alternative Finance where all their research was done. Other sources provided a little quantitative as well as more qualitative data.

2.2 Client-Server interaction

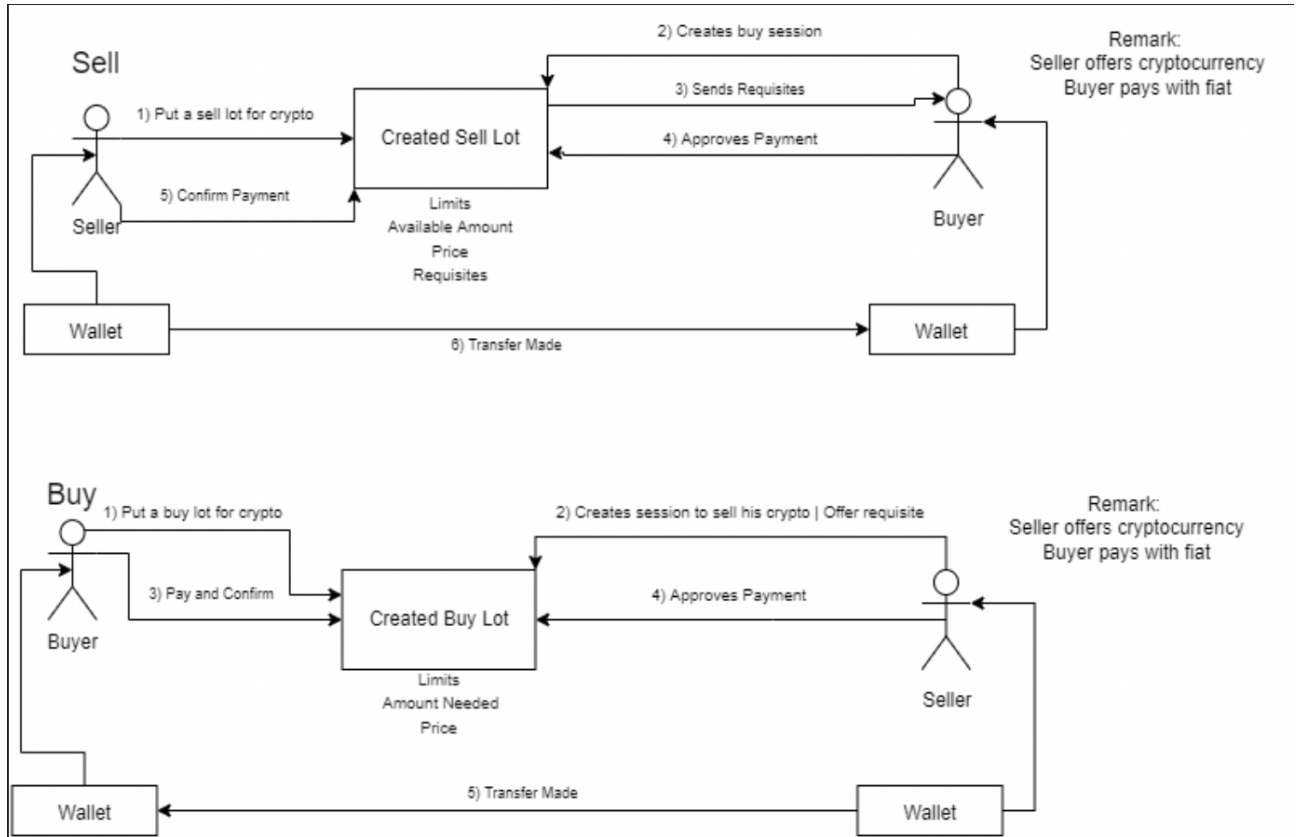


Figure 2.1 – Scheme of interaction

First of all, let's discuss how our project will work and interact between users by describing the figure 2.9. There are two types of lots: *Buy* and *Sell*. User chooses one option and sets a lot to the platform that we developed, and users that want to discover lots for buying and selling can filter the list of lots by their lot type, price, coin type and payment type. After choosing preferred lot, he starts transactional process by sending buying session to the lot service, then lot service returns seller's payment and crypto wallet requisites and the buyer approves payment by sending some request to the lot service while this service sends payment information to the seller, after all the seller confirms payment and the transaction will be completed. When any transaction is completed, a wallet module converts cryptocurrency to another wallet by its address. If one of the traders did not approve their side's confirmation, an appeal module comes in.

2.3 Software architecture

As we described the technology stack in the theoretical part of this document, the project was built in a microservice architecture way because of several causes:

- The project has small modules
- Goal of the project's scaling

- Necessity in new technologies
- Project team's difference in stack choice and their experience

Cryptocurrency transferring service has been divided into six modules each of them as an independent microservice, also there are other connected modules which aim to be a database, cache services as well as Api Gateway that will connect all microservices together and provide interaction between frontend and backend part.

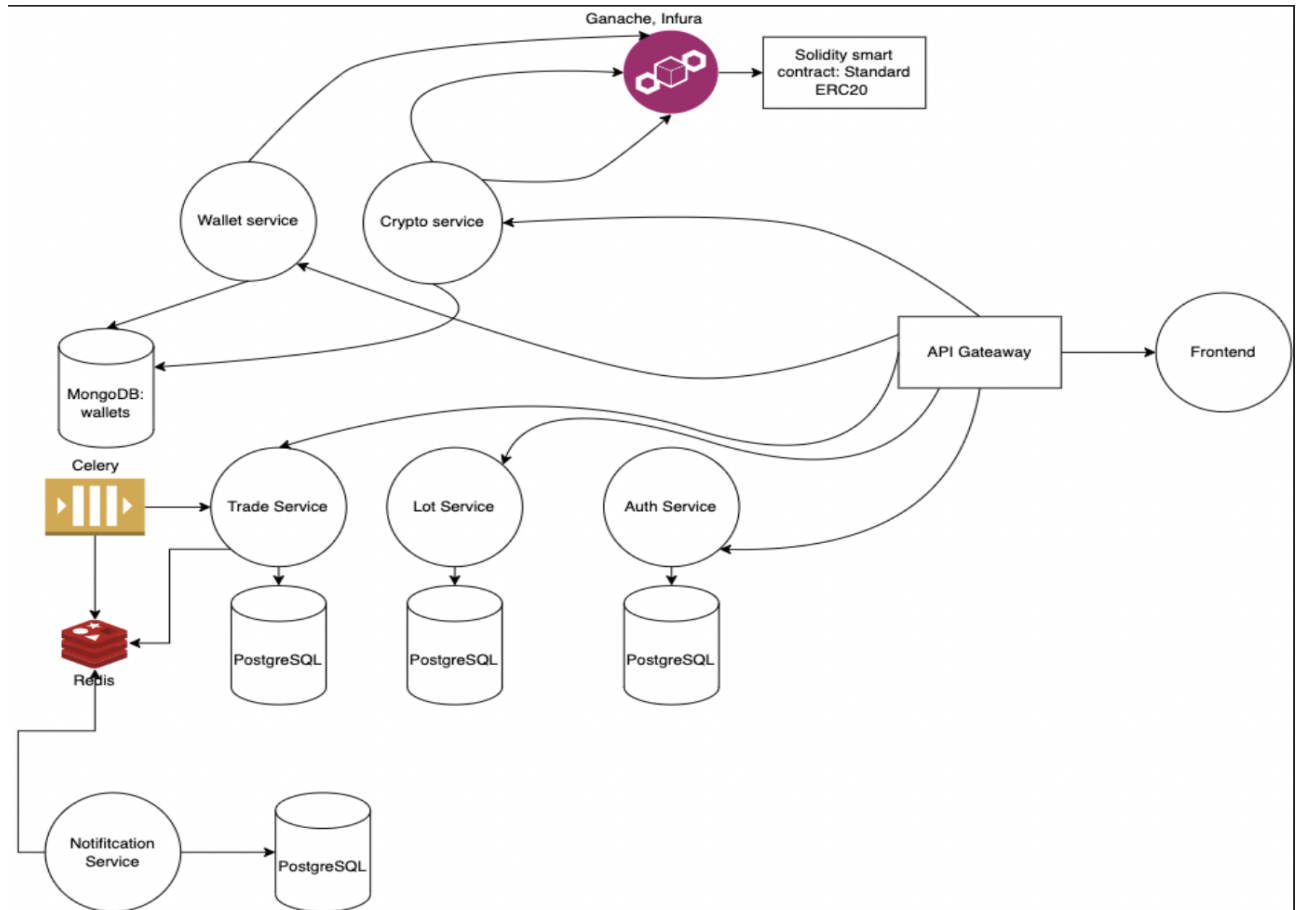


Figure 2.2 – Microservices interaction scheme

The scheme illustrated in the figure 2.2 describes how modules are connected and which databases and caches are used in each service.

2.4 Services description

2.4.1 The Authentication Service

The authentication service collaborates with Postgresql database to store their profile information such as fullname, role, email and password. This module developed using python programming language and FastApi framework. There were several libraries used like SQLAlchemy to work with the database, Json Web Token (JWT) to handle authentication and authorization and other libs. We have chosen JWT due to its high security and performance, because using its access and refresh token which can contain hashed user data, the service can easily transfer this information to other modules that have to manage data about the user. There have been implemented several endpoints to register a new user, authenticate the user, get the user, and update his access token by sending a refresh token.

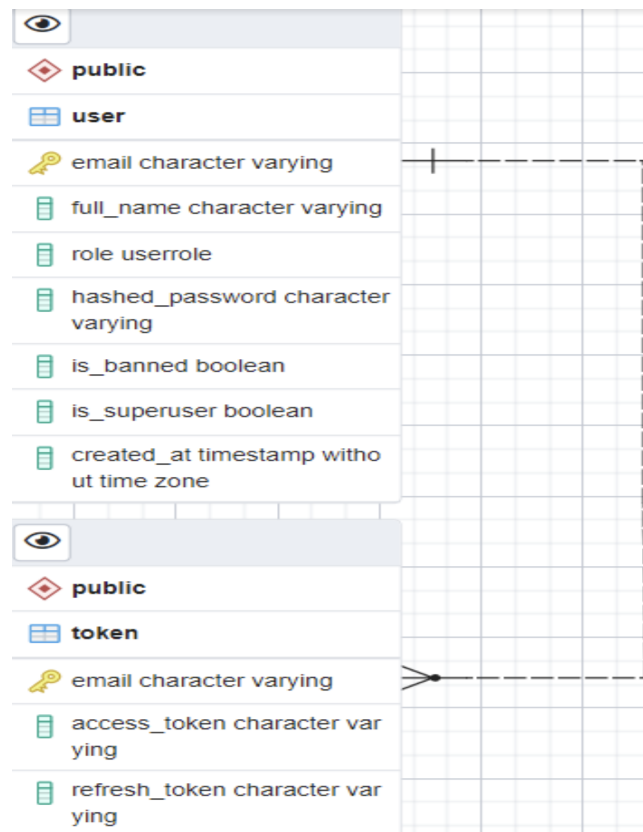


Figure 2.3 – ERD of the auth service

2.4.2 The Lot Service

The lot service aimed to manage the main business logic for creation, updating, deleting and getting lots. In this platform, lot means the trade between the buyer and the seller. There are two types of lots: *Buy* and *Sell*. User can manage his lot which is strictly connected to his P2P wallet. When he publishes his lot, other users who want to buy or sell cryptocurrency, will see this lot and offer

trades to him. After successful trades, however, lot will not be deleted until the amount of funds on seller's P2P wallet will decrease to zero. In addition, this service interacts with the wallet service to get the user's p2p balance by sending requests about his wallet data. Furthermore, there was implemented validation of lots by comparing the user's requested lot data and the obtained wallet data. The service was developed by using python's most famous and useful framework called Django with its included libs as Django Rest Framework, Api Views and ORM which provided workflow with a postgresql database. Basically, Django is the best option for such a simple module that must include CRUD for its models.

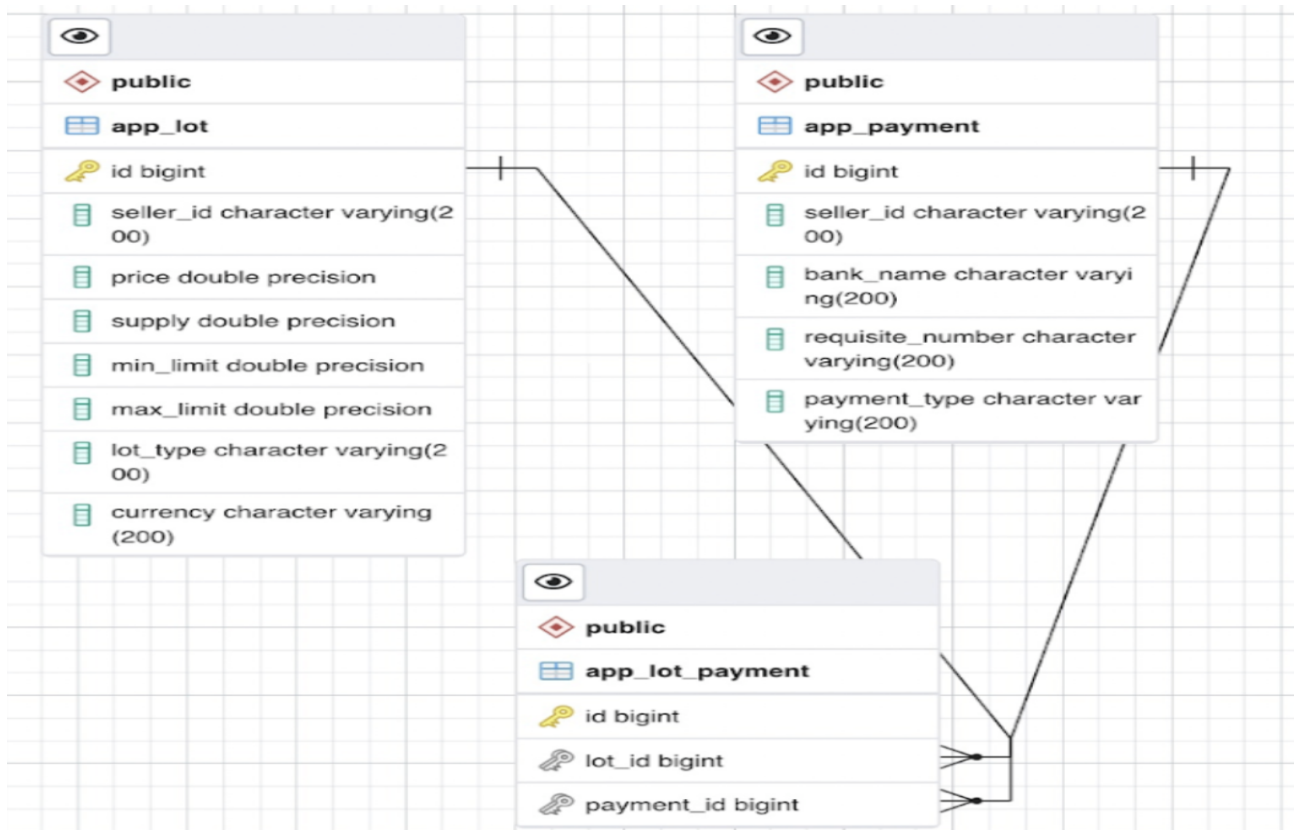


Figure 2.4 – ERD of the lot service

2.4.3 The Wallet Service

The wallet service stands for the management of wallets on our platform. For now, there are 2 types of wallets - Ethereum and ERC20 token wallets. First of all, it is worth mentioning that these wallets share the same Ethereum account, because ERC20 runs on an Ethereum virtual machine and therefore it is fully dependent on the Ethereum account. Moreover, ERC20 represents a smart contract, the function calls of which require gas. Since gas is the small amount of ether, the account already needs to have initial ether on it, and there is no point in creating a new account for this wallet. All in all, Ethereum and ERC20 wallets share the same account. Another important detail is another type of wallet. This is a P2P wallet. More precisely, users generally work with their main wallets. However, they are not permitted to use them on the P2P platform. This is where our P2P wallet is introduced. It is a special wallet for doing P2P transfers of cryptocurrency, and it is strictly defined only on the P2P platform. That means, it cannot be used somewhere else. After doing some transfers, funds from P2P wallets can be transferred back to the main wallet.

Talking about ERC20, a new wallet for it will not be created, because like it was stated before, it depends on an Ethereum wallet, and the created Ethereum wallet can be used for that purpose.

In addition, there is no option of deleting wallets in our platform, since it is practically impossible. More precisely, all the Ethereum accounts already exist in blockchain, and any private key, which follows the rules (*blob of 256 bits; length is not less than 32 bytes*), can generate an Ethereum account with corresponding address.

However, regarding security issues, there may appear some problems during P2P trade between two users. It concerns seller who has amount of cryptocurrency to sell. There is no chance that a buyer of cryptocurrency can deceive seller, since he does the payment by fiat first. Unfortunately, since seller is given some time to approve this payment, he can wait until the timeout of a trade and refusing to approve, thereby deceiving the buyer. Thankfully, there is an option of punishing the seller by freezing his wallet. Actually, it is the option of appeal which leads to freezing, but within the scope of this service, there is an option of wallet freezing.

Wallet service and later-mentioned crypto service works with the above-mentioned wallet models: *EthereumWallet<ObjectId>* - main wallet, *EthereumP2PWallet<ObjectId>* - P2P wallet. P2P wallet inherits main wallet. Wallets contain Id in *ObjectId* format - MongoDB ids, user emails, hashes of user passwords, keystores for encryption of account private keys. Since wallets can be freezed, they also have fields *IsFrozen*, *DateOfUnfreeze*. Regarding remaining fields in P2P wallet, they are just copies of real balance in wallets. These fields are needed in the following scenario:

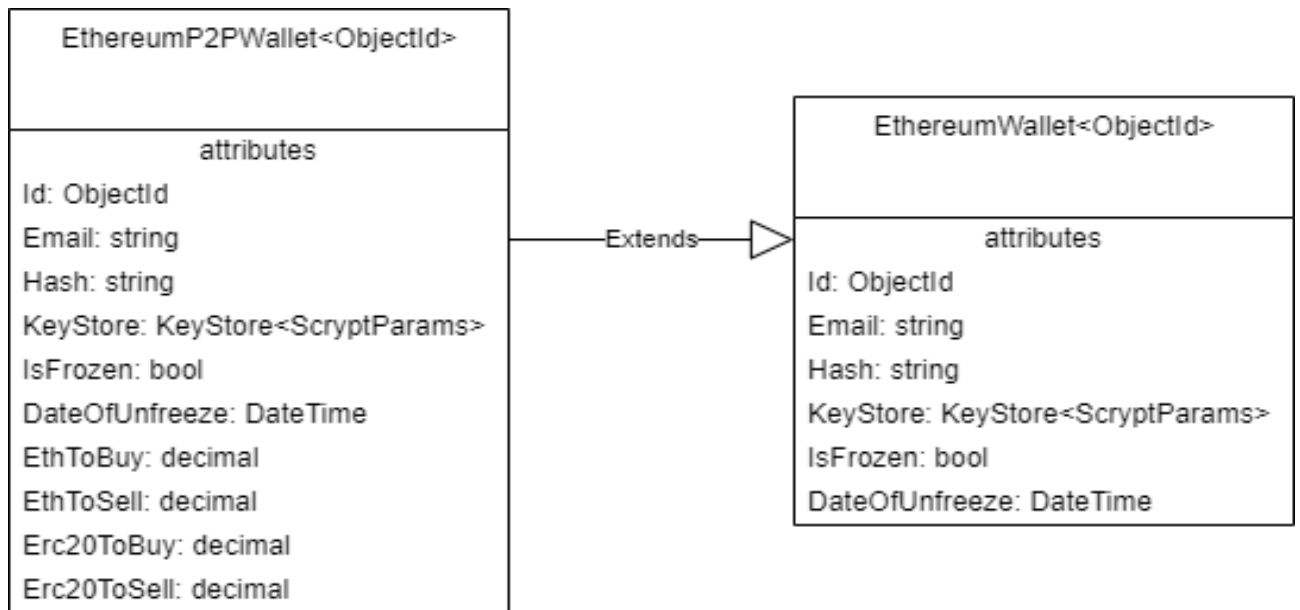


Figure 2.5 – Wallets models

When a buyer wants to trade with seller, he proceeds to lot window and specifies the amount of fiat he wants to pay, and the corresponding amount of cryptocurrency is calculated for this fiat. However, if a user specifies such fiat that the calculated amount of cryptocurrency exceeds the real balance on seller's P2P wallet, then the trade is supposed to fail, since wallet does not contain enough funds. That's why these dummy copies of real balance are used in order to show the limit amount in the wallet.

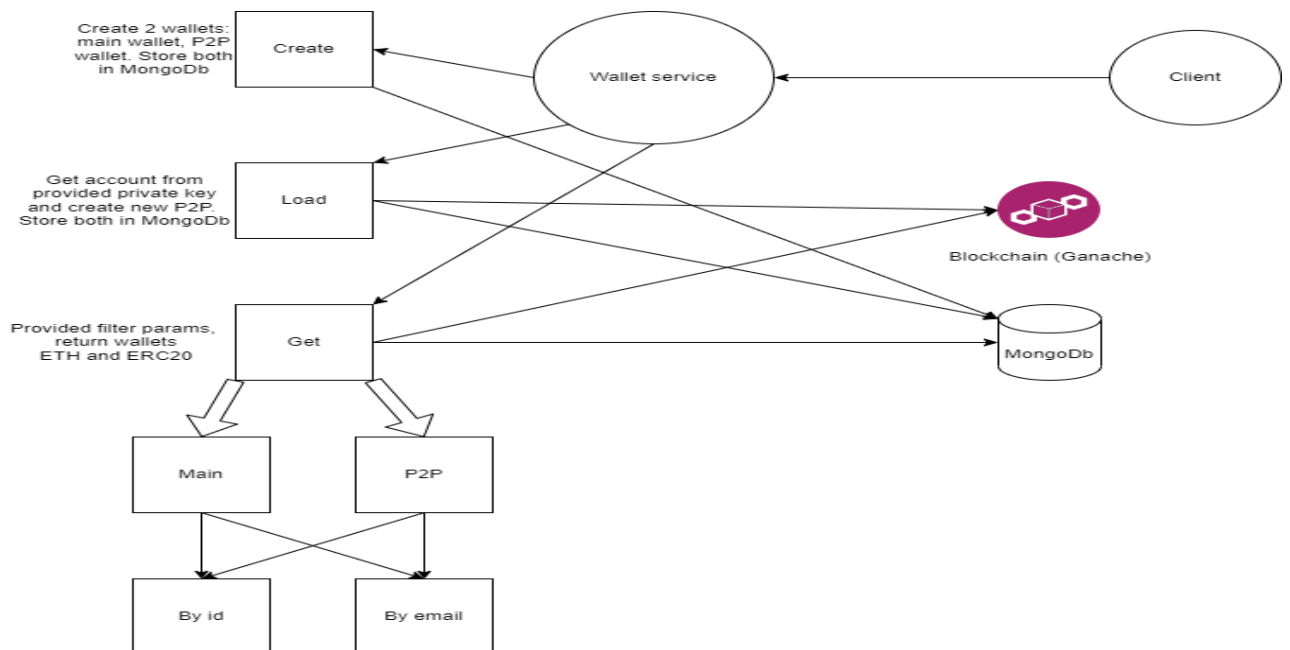


Figure 2.6 – Wallet service scheme

Telling more about the wallet service functionality, it represents 4 main operations: Create, Load, Get, Freeze. Wallet service itself connects to the MongoDB database where all the user wallets are stored, in 2 tables: ***etherWallets***, ***etherP2PWallets***.

Before that, it connects to the blockchain in order to get an existing account or create a new one. As it was written in technology stack section, this service was written using C# as a programming language and ASP.NET Core framework for web development, while the database is MongoDB and blockchain is Ganache. There exist other test blockchains such as Infura, but we stopped on Ganache, because it provides us test accounts with an initial balance of 100 ETH on each. However, we need to interact with blockchain somehow using C. For that we'll use the Nethereum package which is used in both wallet and crypto service. Let us describe each operation more deeply:

- ***Create:***

Create a new wallet. It is assured that the user does not have an existing account. The input fields are the email, password of a user. They are validated. Subsequently, a new private key is generated and correspondingly a new address is also created. Since the private key is in raw format, it needs to be encrypted. Nethereum provides a method of encryption of private key using keystore encryption. An encryption key is the user password's hash. After the keystore encryption of a private key, the wallet is stored with wallet id, email, password hash and keystore, in the database. If everything is done without errors, the successful status code is returned.

- ***Load:***

Load an existing wallet. The user provides email and password, as in Create operation, but also provides his own private key. This private key loads an Ethereum account and validates if it's not a new one. The validation is in the check for emptiness, whether the private key is in hex format and whether the length of the private key is 32 bytes. After that, the same operations are done in Load: private key is encrypted using password hash, then is stored with other wallet information in MongoDB. Finally, depending on the storage, the status code is returned.

- ***Freeze:*** The freeze of the wallets is done by admin only. The input field is the id of a wallet. Since main and P2P wallets share the same id, they are both getting frozen after the operation, and it is logically correct, since user will not be allowed to use his wallets in other cryptocurrency types. After the validation of wallet id by ObjectID format, the wallet with this id is found. Later, the P2P wallet is also found. Since it was stated before, that Ethereum accounts cannot be frozen, there is a trick which is simple. The wallets in database have additional columns called *IsFrozen*, *FreezeExpiryDate*. The first one is a bool flag and serves

as the barrier for user, whilst the freeze expiry date is the date, when the wallet will be unlocked. We have defined that this time period will be 1 month. After that, all the operations with wallet will also be unlocked. Returning to freeze operation, the above mentioned fields are updated, and the status code is returned.

- **Get:** This is a read operation which returns user wallets. They don't expose private key, so only ids and addresses are returned. There is an option of getting main wallets or P2P wallets. They, in turn, are returned either via wallet id or email. Let us consider getting P2P wallet by email:

- 1 A wallet with a given email is found in the database.
- 2 The wallet keystore loads an Ethereum account.
- 3 A balance of an account is retrieved with the help of Nethereum package.
- 4 Response is the wallet id, account address, and balance.

The other get operations are similar to above, except for the operations by wallet id, where we return the same response, but without wallet id.

The choice of technologies for that and the crypto services is conditioned by the following reasons:

- The C# is the general programming language of a developer who wrote these services. On top of that, with the existence of .NET 6 and ASP.NET Core 6, it is extremely convenient to write web services using new features and Minimal Api. Telling more about Minimal Api, it no longer requires to register all services and modify the build of web applications in a special Startup class.

- The chosen database is MongoDB, because the above-mentioned keystores are in JSON format. Since the BSON of MongoDB is similar to JSON, the wallet can be easily stored in collections, without any conversions. Additionally, MongoDB is a NoSQL non-relational database, and it means that the documents are scalable. In the future, it might happen that the wallets will have additional fields, such as account address, or oppositely, lose some fields.

a) Conclusion

The wallet service is a web microservice for managing Ethereum and ERC20 wallets. It interacts with the Ganache blockchain and MongoDB database. The wallet service is responsible for creating, loading and retrieving wallets for Ethereum and ERC20 tokens.

2.4.4 The Crypto Service

The crypto service is responsible for transfers of cryptocurrency to users. In other words, it is the most essential service in the project. There are several types of transfers in our project. Let us describe each one.

- Main to P2P transfer:

This is a transfer from the user's main wallet to his P2P wallet. First of all, users do not have permission to do transfers and generally participate in P2P trades using main wallets. P2P wallet is specially created for such purposes. However, if a user wants to trade cryptocurrency to someone, he needs to have an amount of cryptocurrency on it in order to trade. For that purpose, he needs to transfer some funds from the main wallet to P2P.

- P2P to main transfer:

After the user has done some trades, he has some amount of cryptocurrency on his P2P wallet. Now, he needs to transfer funds back from the P2P wallet to his main wallet. This is done using such a transfer.

- P2P to P2P transfer:

This is a general transfer. When a seller publishes a lot of selling cryptocurrency, a cryptocurrency buyer finds this lot. Then, after the seller confirms payment from the buyer, a P2P to P2P transfer is done. First P2P and second P2P are the wallets of seller and buyer, respectively.

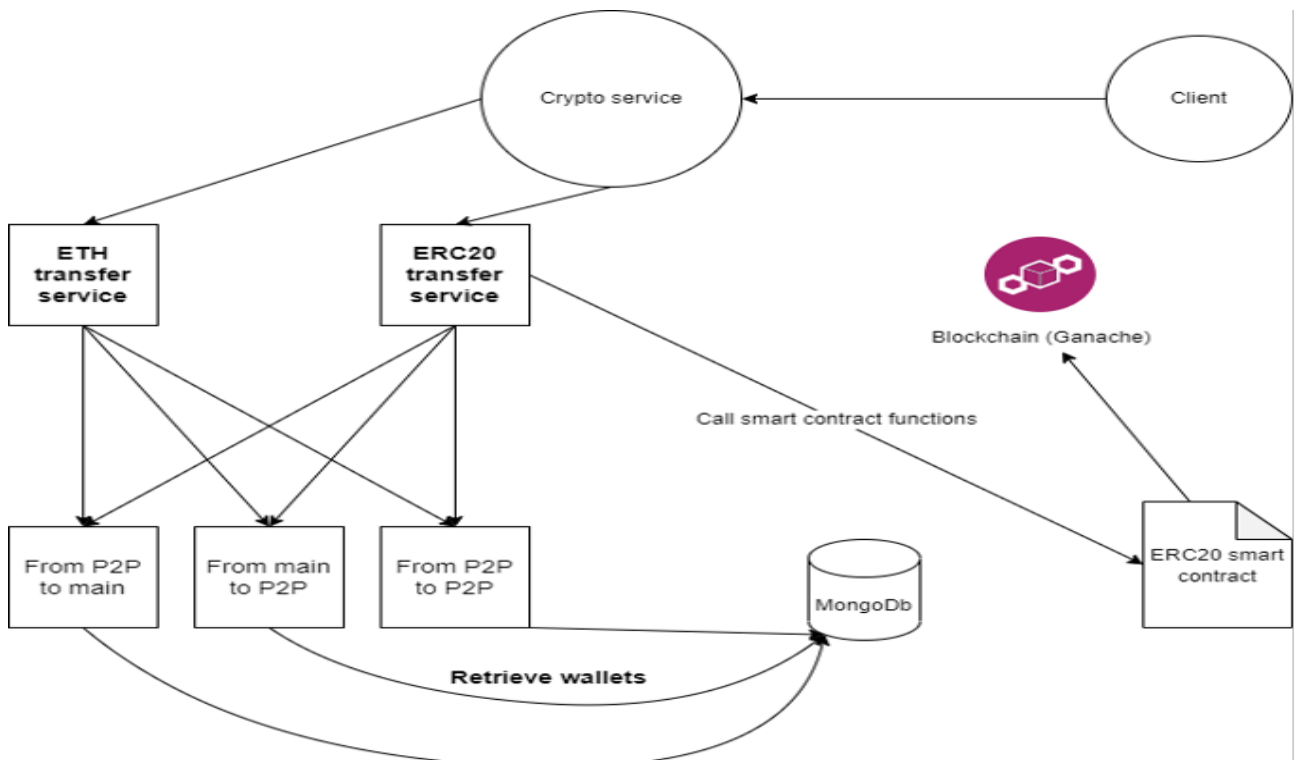


Figure 2.7 – Scheme of crypto service

The crypto service is responsible for transfer operations between wallets. It can provide us transfers of ETH and ERC20 tokens. There are 3 transfers: transfer from main to P2P; from P2P to main; from P2P to P2P. Let us describe each transfer:

- From main to P2P:

This is a transfer for future use of the P2P platform. Like it was stated before, the P2P wallet needs to have some amount of cryptocurrency in order to publish lots for selling. The input fields are walletId, amount. The main wallet with the given walletId is found in the database, and then the P2P wallet is also found. Main wallet and P2P wallet share the same wallet id, so it is easy to find it. Then, an account is loaded from the keystore of a wallet using password hash and JSON string of the keystore. After that, the loaded account signs the transaction and transfers funds to the P2P wallet's address. The transaction receipt indicates the success of the transfer operation. It is noteworthy that the transfer operation requires gas, so the main wallet needs to have some ethereum for paying it.

- From P2P to main:

This operation is similar to main to P2P transfer, except for changing roles. Now the P2P wallet is retrieved as well as the main wallet, and it transfers back to the main.

- From P2P to P2P:

Here, the input fields are walletId, recipientId, amount. This transfer operation is done by the end of the trade session, where the seller approves the payment of the buyer and the funds are successfully transferred. Firstly, the full P2P wallet of the seller is retrieved from the database, while only the address of a wallet of the buyer is also retrieved. P2P account signs the transaction and transfers to the P2P wallet of the buyer. Again, the result of the operation is the transaction receipt.

These transfer operations work with ETH. However, as it comes to ERC20, there is a slightly different approach. Generally, when working with ERC20, we interact with smart contracts which are deployed in blockchain and written in Solidity. For now, we work with StandardERC20 tokens, but there will be an opportunity to work with other tokens as well. There are functions in smart contract. In terms of transfer, we need to know two operations only: balanceOf(address account), transfer(address recipient, uint amount). First operation is used to validate the amount of tokens that we want to transfer, or whether it does not exceed the input amount. The second one is the actual transfer, we call it as a function call using the same Nethereum package.

a) Conclusion

The crypto service is a web microservice which is responsible for transfers of ETH and ERC20 between user wallets. It uses practically the same technology

stack as wallet service, except for additional technology Solidity which allows us to write smart contracts on Ethereum. This is where the ERC20 smart contract was introduced, and we interact with him using function calls which are comfortably decorated and provided by the Nethereum package. The service lets us transfer funds between wallets using 3 following types of transfer: from main to P2P, from P2P to main, from P2P to P2P.

2.4.5 The Trade Service

The trade service is the main engine of the whole system that works as a decorator upon the “Crypto Service”. It is strongly coupled with all available services of the project. The logic of this service is initially designed as P2P with the guarantor where the guarantor role is played by our service, however, this idea was dropped due to the high cost of the network gas that could lead to big losses.

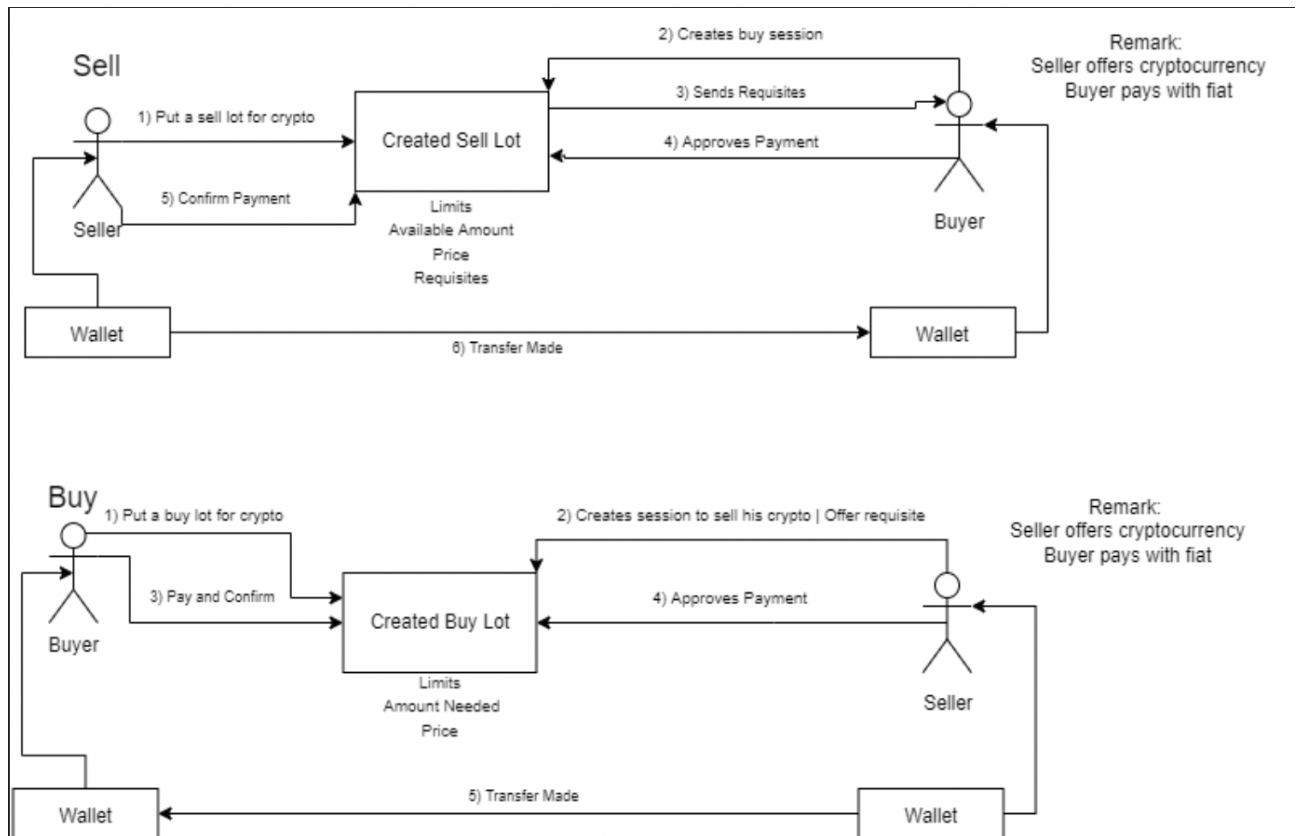


Figure 2.8 – Scheme of interaction

The figure above illustrates to us the two variants of how the trade will proceed throughout our platform without any fear for the end-users. These variants are symmetrically the same and the main difference is in what lot type has been chosen.

- The first subscheme demonstrates the trade process flow upon the sell lot type. That means, that the lot was created with the reason to sell crypto for someone. The crypto owner puts a sold lot with some fixed crypto amount and in-lot limits for the buyer. On the other side, the buyer creates a buy session. The amount to buy will be reserved for him during all the sessions or till the session will expire. Lot service sends the payment information to the buyer, so he could make a payment on the seller's bank accounts or other payment services. After payment, he could push the button that the payment from his side is succeeded.

The seller should check the truthfulness of the buyer's approval and make approval after the fiat payment reaches his payment system. The problems at this stage will be discussed further. When both sides have approved the trade details, the transfer of crypto goes under the hood of the out platform.

- The second subscheme shows the vice-versa scenario. Now, the lot is placed for buy, so the lot producer became one who needs to pay and approve payment firstly. Then the flow goes as in the first subscheme.

The worth mention thing is how the buyer will gain his crypto if the seller will stay away from pressing the approve button. For such a situation, the appeal service was created, so the platform administrators could prevent such inconvenient troubles.

The module consists of one entity named Transaction that contains several significant fields like seller/buyer information, transaction information (amount of crypto, fiat amount, the status of the transaction (CREATED, ON_PAYMENT_WAIT, ON_APPROVE, SUCCESS, CANCELED, EXPIRED) and others (Figure 2.9). This information is also accessible for users appealing to their permission/role rules.

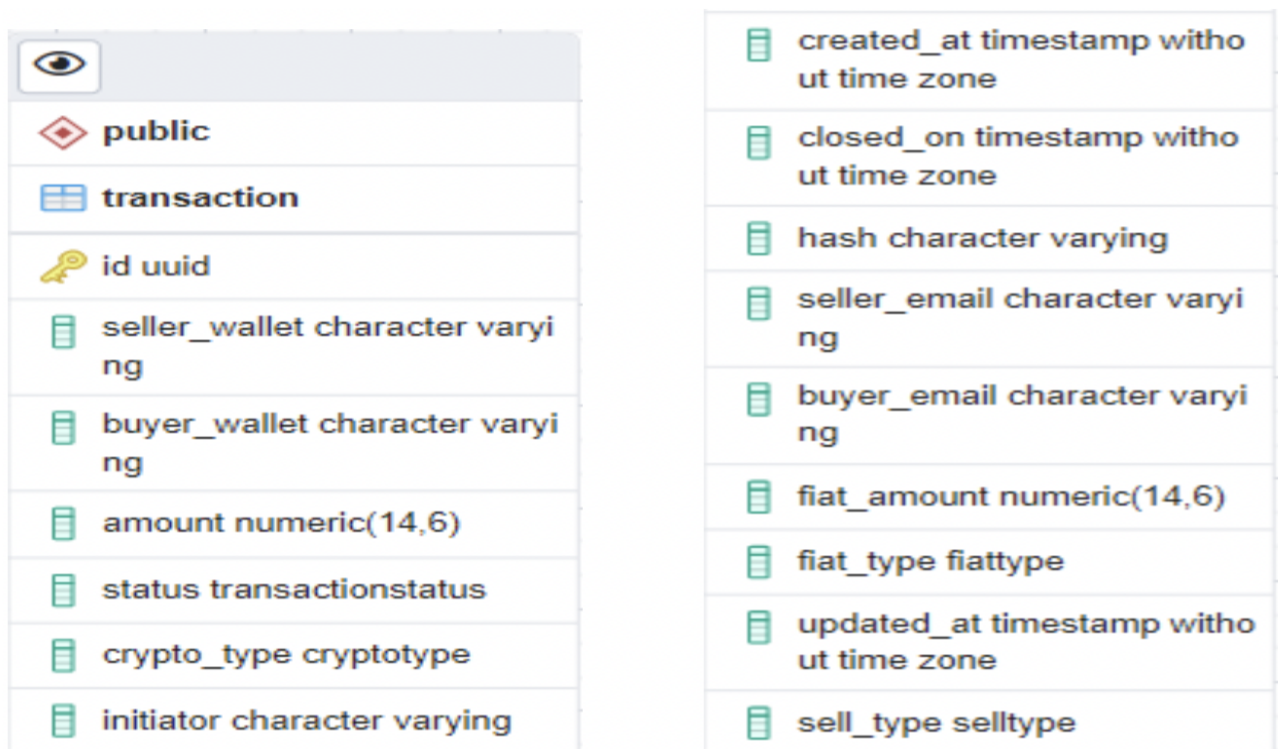


Figure 2.9 – ERD of trade service

To fully understand the trade service, it's necessary to understand how Crypto Service and Wallet Service work. Looking under the hood of the Trade Service, it's clear that it uses its endpoint to work with users' crypto assets. As an example, after the creation of a lot, the user raises his/her shadow supply account (for buy

or sell). In this example, the lot placed for sale will be considered. After that, the buyer will create a session that will decrease the seller's supply to avoid race conditions in the seller's assets and to create an illusion of the guarantee. During the trade, statuses will change several times without other services intervention. Furthermore, every status change triggers the notification service (which will be described further). On SUCCESS status the transfer in Crypto Service will be triggered between end-users P2P accounts. On CANCELLED or on EXPIRED statuses the supply will be rollbacked. The trade service also uses delayed tasks, using Celery distributed queue, to expire the trade in certain conditions (when the first payment isn't made in some amount of time). The full scheme of the service is shown below:

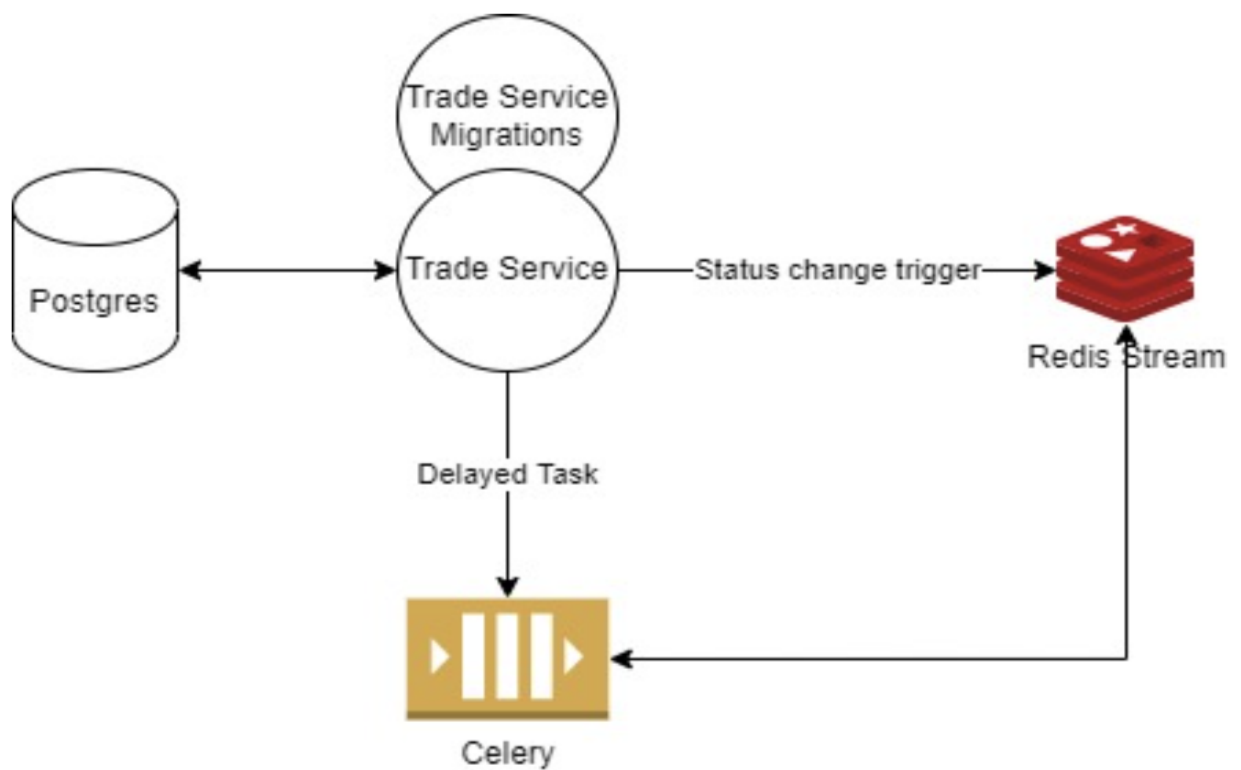


Figure 2.10 – Trade service scheme

a) Conclusion

The Trade Service is a web microservice that is responsible for trade logic and communication with crypto services. It uses practically the same technology stack as an authentication service, except for some specific technologies like Celery and Redis. These are used for communication with the Notification Service and creating delayed tasks. The trade service is responsible for creating in-app transactions, regulating them, and give information about existing ones.

2.4.6 The Notification Service

The Notification Service is a microservice that is responsible for managing notifications throughout the platform. For now, it only works for the transaction status change, but it could be easily extended for other notification types.

How it works?: The Notification service works on the trigger that originates from the Trade Service. The fast connection between them was established with help of a message broker, in this situation we have chosen Redis, and its new data type Redis Stream. As in Kafka, it allows us to create consumer groups to easily consume named notifications. This also allows us to easily extend the application with other types of notifications. In brief, the Trade Service produces (adds) data to the named stream and the consumer launched on the base of the Notification service periodically listen's to declared streams and writes data to its own database for further interaction.

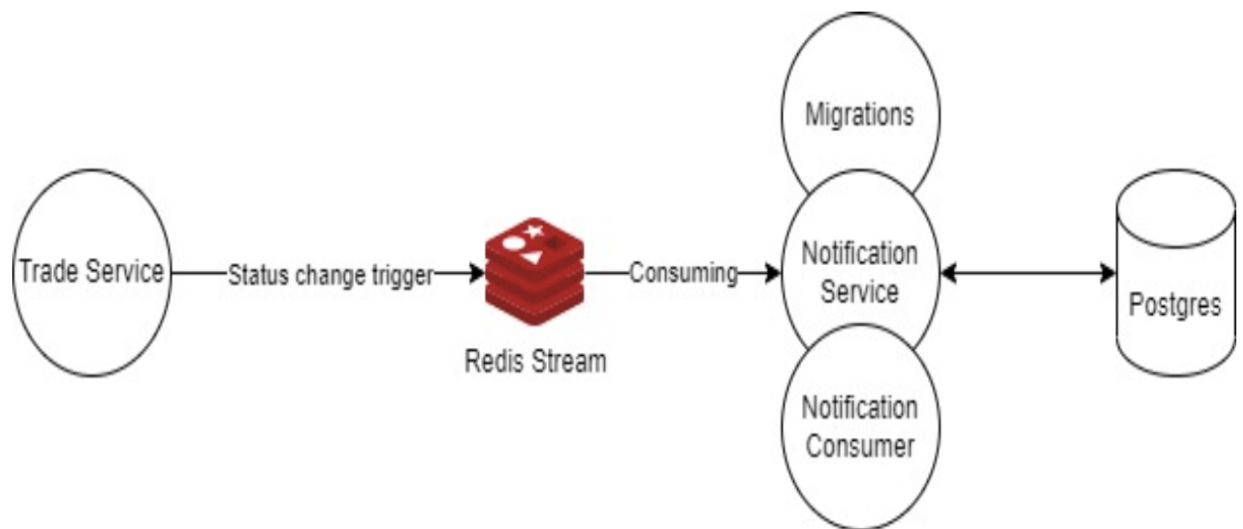


Figure 2.11 – Notification service scheme

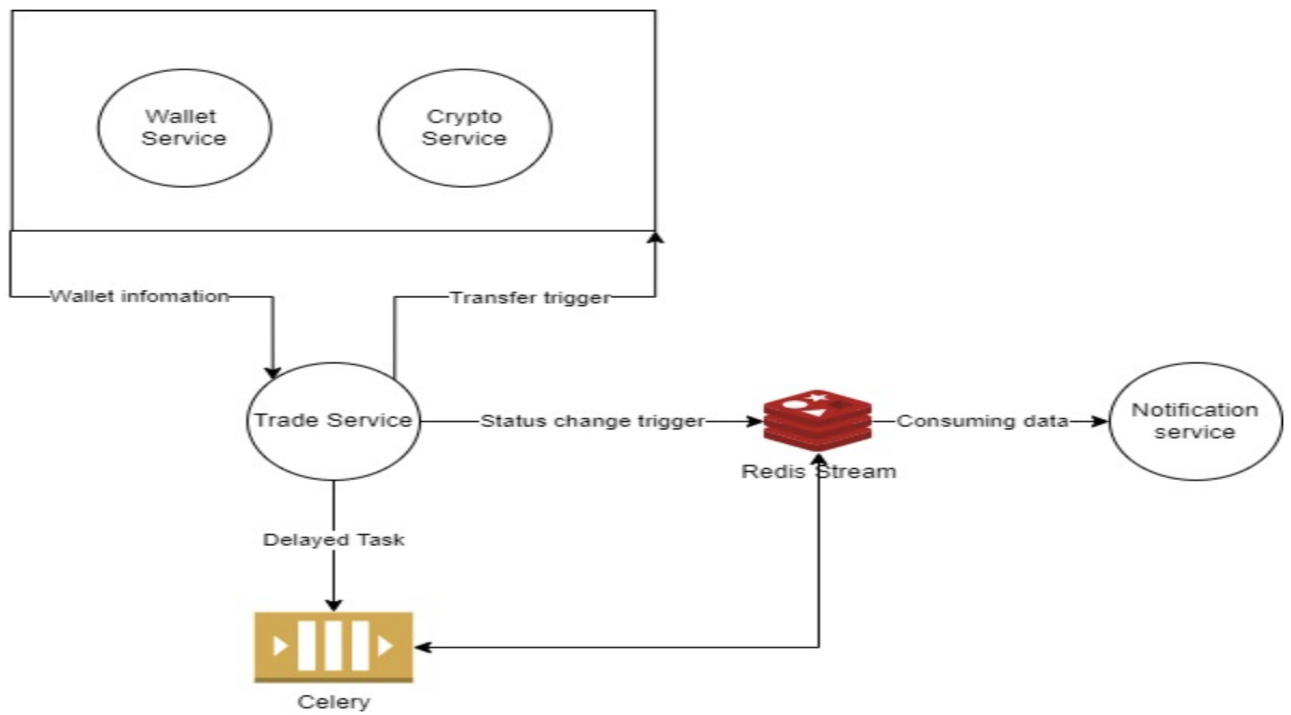


Figure 2.12 – Trade and Notification Services interaction

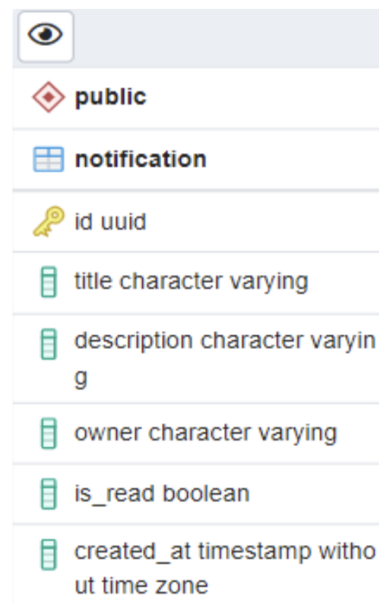


Figure 2.13 – ERD of notification service

On the schemas above, the interaction between Notification and Trade services is illustrated. The main communication, as it was described before, was established via Redis.

The module has one entity named Notification that consists of several fields, namely id, title, description, owner, is_read, and create_at. These fields are

enough to fully inform users about the events that proceeded in the background. The stack used in the Notification and Trade service is practically the same with some differences. As a backend language, the FastAPI framework of Python was chosen. The choice of FastAPI is obvious because services must be fast and lean. Django is a great framework but it doesn't provide asynchronous functionality out of the box. It can't support many requests because of database connection logic under the hood, so you need to add extras like PgBouncer. As a database management system, our choice is PostgreSQL because it's a popular, powerful, and well-known database management system. For scheduling tasks, in our case expiration of the trade, the option of Celery is beyond question. Celery is great for delayed tasks and could work with Redis that we already have in our system. Redis is not used as a mem-storage but as a message broker. Redis and Python are well compatible with help of the redis-py module.

a) Conclusion

The Notification Service is a web microservice that is responsible for notifications. It uses practically the same technology stack as an authentication service, except for some specific technologies like Celery and Redis. These are used for communication with the Notification Service and creating delayed tasks. The trade service is responsible for creating in-app transactions, regulating them, and giving information about existing ones.

2.4.7 Appeal service

The appeal service is responsible for creating, retrieving and deleting appeals. Appeal is submitted by the user in case he thinks he got scammed during the trade session. In P2P trades, there are always people who try to scam you, but there is an opportunity to make an appeal. However, making an appeal requires some kind of proof. Otherwise, it will be considered invalid. User can attach a payment receipt in order to confirm that he did really pay money to the seller.

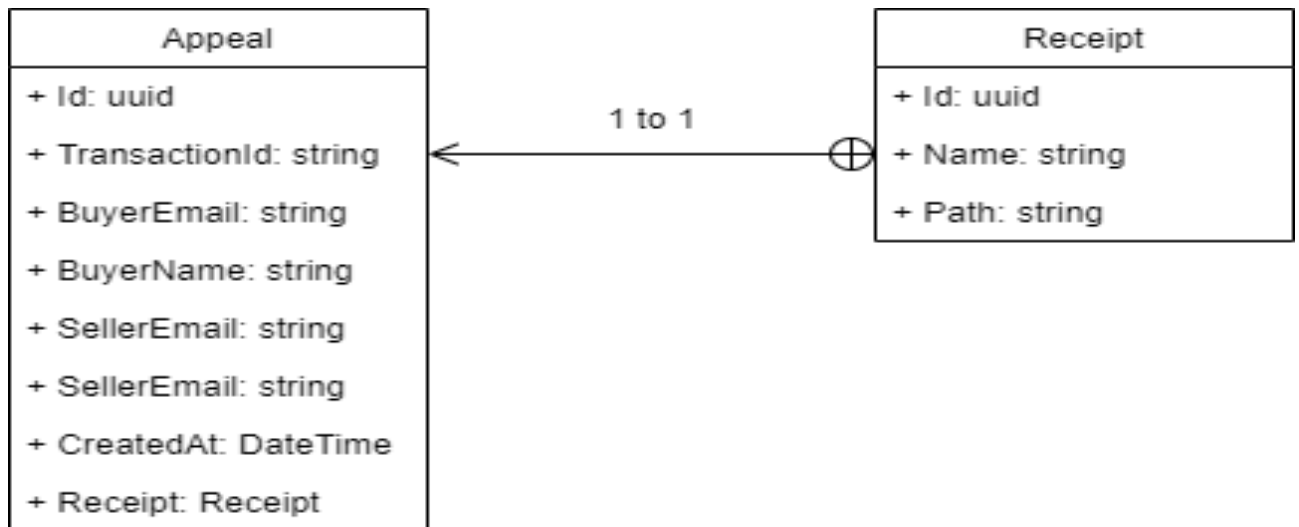


Figure 2.14 – Appeal and Receipt models

There are 2 models inside the appeal service: **Appeal**, **Receipt**. Receipt has 1 to 1 relationship with appeal, and can be loaded when querying appeal. It has the name of a receipt, and path to actual receipt. These receipts come as *.pdf* files, and are physically stored in the application folder for receipts. Regarding appeal, it has all the information about the appeal sender, seller or potential scammer, attached receipt, and the date of creation.

Both models have *unique identifiers* which come as GUID, and the receipt is connected to appeal with cascade option which means that with the delete appeal the associated receipt will be consequently deleted from tables also. The *lot id* means the id of a session (transaction) between buyer and seller. It is needed so that tech support could have reviewed the details of a failed transaction and take the corresponding measures.

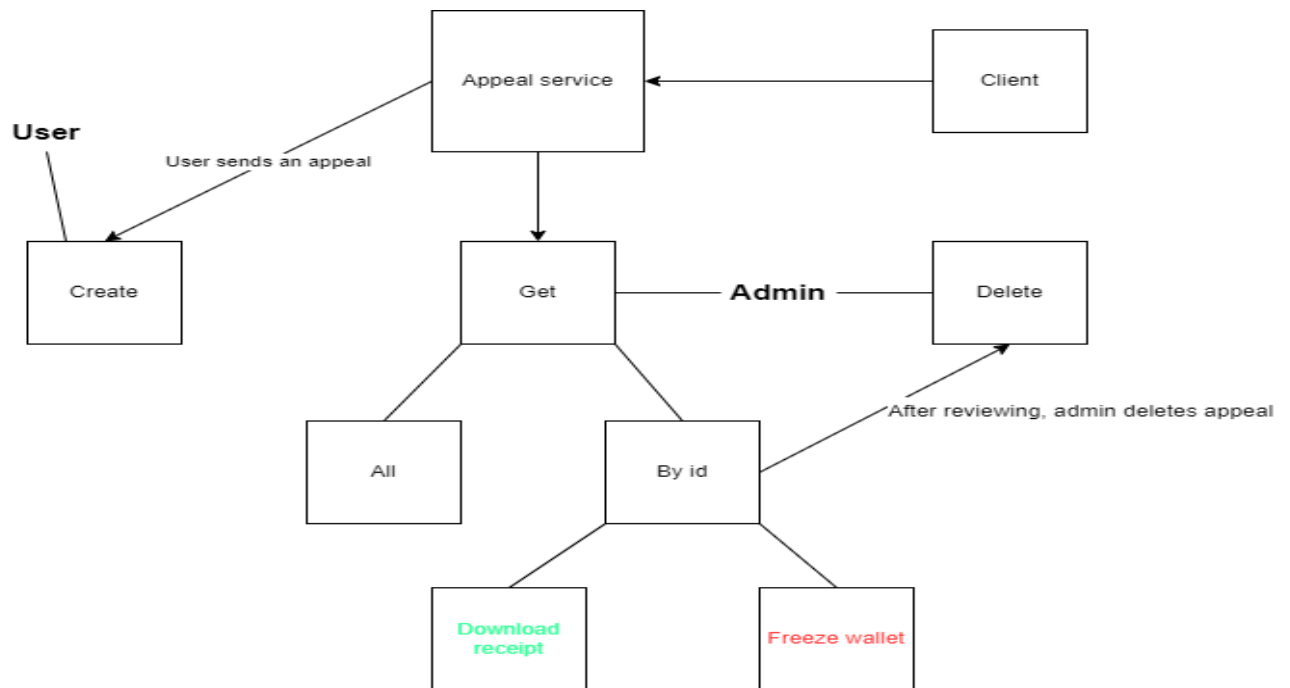


Figure 2.15 – Appeal service scheme

Appeal service has all the CRUD operations for the appeal model, except for update. Generally, only the admin can use the appeal service: there will be created a special page for tech support, the role of which is admin. However, creating an appeal can be done by the user. Nonetheless, let us describe each operation in the service:

- ***Create appeal***

User creates an appeal. This operation is called when the user (cryptocurrency buyer) suspects the seller in a scam attempt and sends an appeal request in the special window in the website. The appeal contains the name and email of a buyer and seller, the id of the lot (transaction), while the date of creation is calculated on the server side. Then, on the server side, the input appeal is validated. Next, according to provided emails of buyer and seller, service sends an HTTP request to user service in order to retrieve users with these emails and get their full names. The new appeal is created with users information and another data, and then stored in the MySQL database.

- ***Get appeals***

1 *All appeals:*

It's a simple return of all appeals contained in the database. However, not all fields are returned, but only emails of buyer and seller, date of creation. For performance and user experience purposes, the return of appeals is in paginated format (SQL TAKE and SKIP queries). That means, that admin can scroll through appeals using pagination.

2 *Appeal by id:*

After clicking on some particular appeal, the return of this appeal by id will be next. Here, the admin will see the full information about the appeal, including the opportunity to download a receipt and to freeze a wallet. When clicking on specific appeal, it is retrieved from the database. However, there is one additional detail there. After retrieving from database, service sends HTTP request to transactions API in order to get the details of a transaction. After everything is done, the ready data is returned.

- ***Download receipt***

Download receipt will return a receipt file, stored in receipts folder, which was submitted by the user during appeal creation and is associated with this appeal. The receipts come from appeal creation in *.pdf* format. That convention is established, since the image format receipts may be changed by third-party programs, while the fiat payments usually come in *.pdf* format, such as in Kaspi or Halyk bank.

- ***Freeze wallet***

The admin will have an opportunity to freeze the wallet of a seller. This should be done carefully, as the operation is non-revertible and the wallet will be unfreezed after some period of time. The admin will take a decision after careful review of an unsuccessful trade and verification of scam by the seller. Since the wallet freeze is done in the wallet service, the appeal service sends HTTP request to that service and the wallet is consequently frozen. After that, there is no need to review this appeal, so the tech support is free to delete this appeal.

- ***Delete appeal***

This operation is understandable. The input field is the appeal id only. After finding a matched appeal with the same id, it is deleted from database. On the client side, the tech support will be redirected back to all appeals page after delete operation.

The chosen database in this service is MySQL because of particular reasons:

- MySQL is used for its speed and performance. It offers memory caches, advanced indexes and other performance-related features. In addition, it is not overloaded by extra features and therefore works faster.

- MySQL is easy to configure. It is easily configured and can be deployed easily as well.

- MySQL is user-friendly. In other words, it will not take much time to learn MySQL, or only needed features in it.

a) Extra detail about service

It is noteworthy, that the contact with buyer and seller in order to notify them about the made actions is done by email notification. This is where the mentioned package MailKit is used. After the wallet is freezed, the appeal service next notifies buyer and seller about appeal results. The email messages are constructed and sent from predefined email to the email addresses of both users. Before all of that, the tech support email was configured and given access to SMTP server. Now, after the email messages are built, the new SMTP client instance is created, the tech support email connects to it, authenticates using the given credentials, and successfully sends formulated messages to recipients.

b) Conclusion

Overall, the appeal service represents the web service which is responsible for creating, deleting and retrieving user appeals. Regarding the appeals themselves, after their careful review and associated transaction details, tech support either ignores appeal or freezes seller wallet and notifies by email both sides.

2.4.8 Chat service

During the trade session, both buyer and seller can communicate with each other using chat. However, for viewing new messages, they can't reload the page any time. In other words, they need to send and receive data in real-time somehow. Chat service is responsible for that by providing real-time communication between server and client. Here, the general technology for the service is web sockets which allows it to work in real-time. The chat service itself is the web server which uses *SignalR* and *Redis* cache. Here will be explained the use of each technology:

- 1 SignalR is a technology owned by Microsoft which allows the use of remote procedure calls (**RPC**) using the communication principle of client-server. In chat service, this technology allows the return of data in real-time format. On the client side, SignalR can be used again as a JavaScript library which can start a connection and send RPC messages to the chat service.
- 2 Redis is a **NoSQL** key-value storage which stores lot identifiers as keys and users as values. It is used for its principle of storage in memory and high performance as a result.

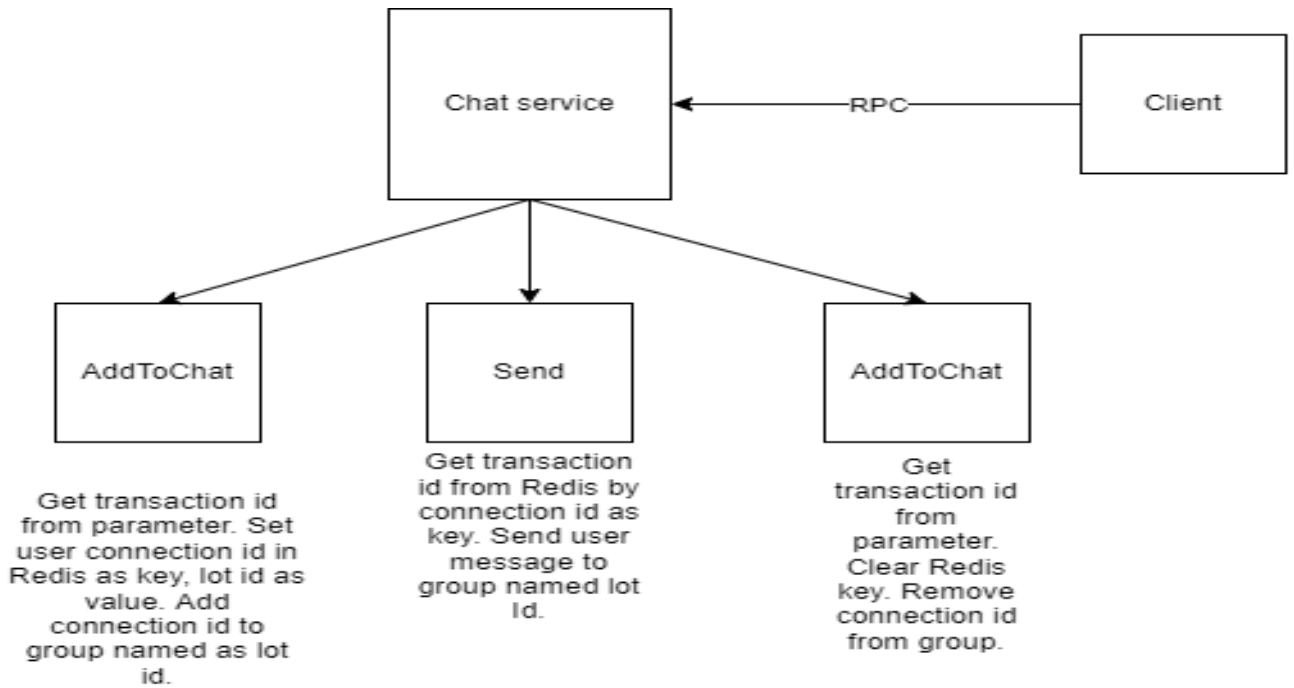


Figure 2.16 – Chat service scheme

Chat service has 3 operations: *adding to chat*, *removing from chat*, and *sending messages*. Let's describe each operation:

- ***Add to chat***

Adding users to chat. In SignalR, we can identify users by their connection ids which come from the client. The input parameter is a lot id. Then, Redis stores connection id of seller and buyer as keys, and lot id for both as value. After that, we use a special method provided by SignalR, which adds users to a group which can be named by lot id. This is done, so that only users in this group can view each other's messages.

- ***Send***

Sending messages from users. We take again connection ids from clients and get a lot of id from Redis by providing connection ids as keys. Then, we send an input message from the user to the group with the lot id, so that users in the group can see it.

- ***Remove from chat***

Removing users from chat. We clear Redis keys by users connection ids and associated lot id. Then, we call again a special method from SignalR, which deletes this group.

Chat service heavily interacts with client side. The point is that the connection is established on client side by using *HubConnectionBuilder* class in JavaScript. It starts a connection and then invokes the methods defined in the chat service itself, passing the parameters as well.

a) Conclusion

Chat service is a light-weight service, which allows us to use real-time communication between server and client for chatting purposes. It uses an abstraction technology on web sockets, called SignalR and memory cache Redis. On the client side, the client library SignalR is used to start a connection to a chat service and send remote procedure calls to it.

2.4.9 API Gateway

The final service in the project is API Gateway. Its main responsibility is to redirect the incoming requests to client and pass them to corresponding services. It therefore interacts with all the available services via REST. The communication with other microservices is done through special routes defined in API Gateway.

Since the package for gateway is Ocelot, the route configuration is set in the *'routes.json'*. There are defined upstream and downstream paths (microservices and client), HTTP methods, hosts and ports. Basically, this is everything that is needed from the gateway. However, there is an additional functionality which is related to authorization of users. It is done through JWT role-based authorization. For defining roles, the role headers are set in route configuration as well. The authorization, in turn, is specified in the special middleware.

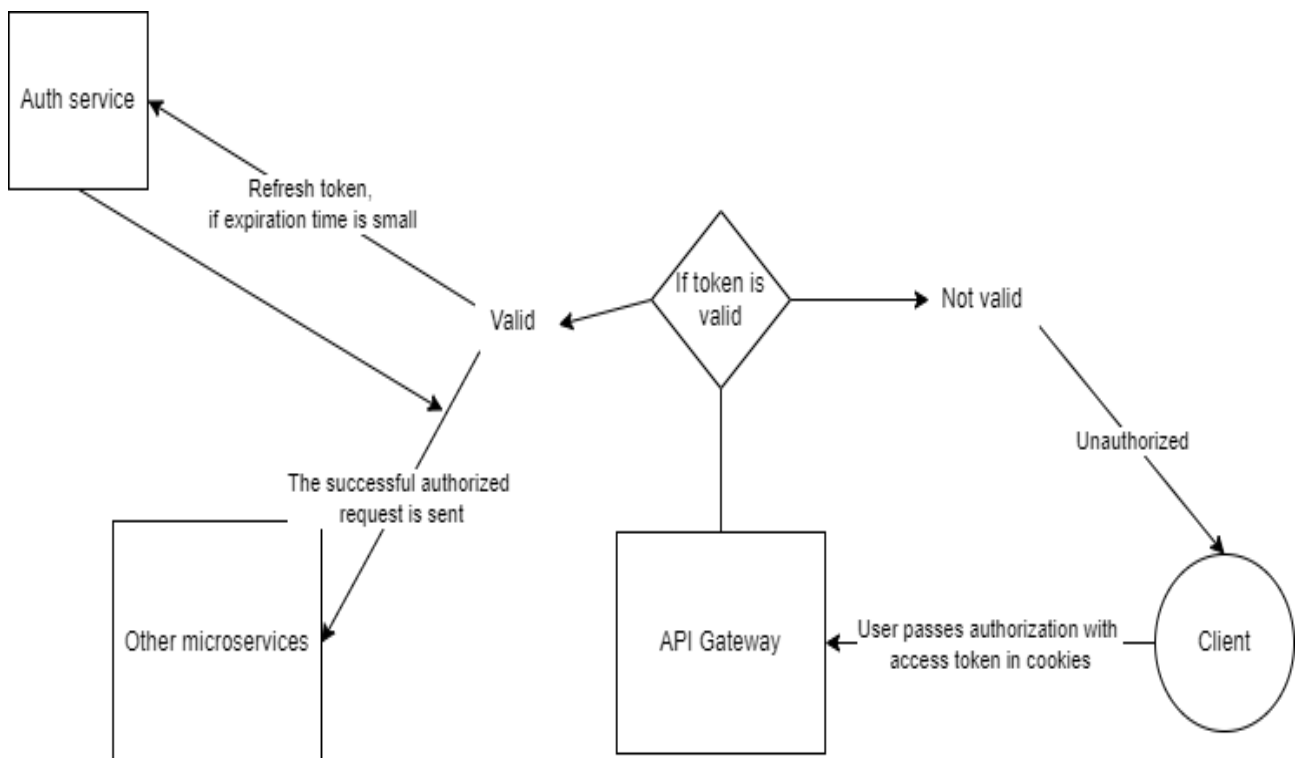


Figure 2.17 – API Gateway scheme

The figure 2.17 describes the flow of a request from client to requested resource. Everything starts from the authorization. The order of the authorization steps is

given below:

- 1 Check whether the role header is present in incoming request: if not present, then it is a open access resource and everything is fine.
- 2 Retrieve JWT access token from cookies and check if it is empty.
- 3 Validate the access token by checking if it is readable.
- 4 Verify the token signature in the access token.
- 5 Get role and expiration date from the token: if everything is valid, then the request is valid and request to upstream microservice is sent successfully.

If any error occurs during token validation, then the request is considered unauthorized and the corresponding status code is returned.

If everything is valid, but the expiration left time is small, then the request to authorization microservice is sent. It is done for refreshing token. After successful response, code is just executed further.

a) Conclusion

All in all, the API Gateway service is a light-weight web service which is responsible for redirecting requests to microservices and previously authorizing users and refreshing token in case, if token will expire soon.

CONCLUSION

Our investigation was about cryptocurrencies and their marketplaces. By the results of this research implemented a brand-new cryptocurrency transferring service which will cover all the highlighted points from this research paper. In conclusion, the topic of this work has significance for particular people nowadays according to the research results, and they have a necessity of a new trading platform. At the beginning we formulated the strong goal which we need to attain: ***To develop a cryptocurrency transferring service according to drawbacks of existing solutions and their best practices in order to use the best methods that are presented nowadays with our additional improvements and algorithms.*** To follow our aim there have been several objectives created.

Objectives acquired:

- Find out the current ways of crypto transferring
- Define the main problems and its possible solutions of the current transferring methods
- Evaluate the best practices and technologies related to crypto transfer theme
- Develop a trustful and easily accessible system for cryptocurrency transferring

According to the results of our first objective, the theoretical part shows us that there are 3 main alternatives. The majority of crypto owners use brokerage and trading services like **Binance, Paxful and Blockchain.com**. In addition, people want to trade Ethereum, Tether and Bitcoin due to their high values in the speculation purposes first and gain financial freedom from banks. Alternatives have their own pros and cons.

The second objective was covered by the cons of current solutions as low anonymity, centralized system, private keys stored in their servers, difficult for beginners, mediocre customer support, and service withdrawal fees.

The third objective was passed by the pros of the current solution such as:

- 1 High security
- 2 P2P trades
- 3 Low transaction fees
- 4 Built-in exchange functions

Afterall, the fourth objective which was aimed to develop a cryptocurrency transferring service was successfully completed and described in the practical part of this paper.

2.5 Further research

The developed cryptocurrency transferring service is not ideal. There is a big necessity to provide additional research by providing surveys and expert interviews which will help us improve the trading platform. Hopefully, our project's architecture follows new standards such as event-driven microservice architecture and new technology stack that can help us to scale up the project in the future. We are planning to add support for other crypto coins, develop a mobile version of the application and improve the quality of the service.

BIBLIOGRAPHY

- 1 *Farell, R.* An Analysis of the Cryptocurrency Industry / R. Farell // *Penn Libraries, University of Pennsylvania*. — 2015. — P. 4.
https://repository.upenn.edu/cgi/viewcontent.cgi?article=1133context=wharton_research_scholars.
- 2 *Scott, B.* Glossary of decentralised technosocial systems: Cryptocurrency / B. Scott // *Internet Policy Review*. — 2021. — Vol. 2.
<https://doi.org/10.14763/2021.2.1561>.
- 3 *Jodet, L.* Differences Between Centralized Exchanges (CEX) and Decentralized Exchanges (DEX) / L. Jodet // *Medium*. — 2021.
<https://medium.com/arianee/differences-between-centralized-exchanges-cex-and-decentralized-exchanges-dex-517df354dfa8>.
- 4 *Dylan Yaga, P. M.* Blockchain Technology Overview / P. M. Dylan Yaga // *Gaithersburg: National Institute of Standards and Technology*. — 2018.
<https://doi.org/10.6028/NIST.IR.8202>.
- 5 *Rauchs, D. G.* GLOBAL CRYPTOCURRENCY BENCHMARKING STUDY / D. G. Rauchs // *Cambridge: The Cambridge Centre for Alternative Finance*. — 2017. <http://dx.doi.org/10.2139/ssrn.2965436>.
- 6 *Abeer ElBahrawy, L. A.-S.* Evolutionary dynamics of the cryptocurrency market / L. A.-S. Abeer ElBahrawy // *The Royal Society Publishing*. — 2017.
<https://doi.org/10.1098/rsos.170623>.
- 7 *Richards, Mark.* Software Architecture Patterns. Understanding Common Architecture Patterns and When to Use Them / Mark Richards. — 2015. — Pp. 1 — 45. <https://blog.mrjb.me/wp-content/uploads/2021/10/Software-Architecture-Patterns.pdf>.
- 8 MongoDB database documentation. — <https://www.mongodb.com/docs/>.
- 9 ASP.NET Core documentation. — <https://docs.microsoft.com/en-us/aspnet/core/?view=aspnetcore-6.0>.
- 10 Nethereum documentation. — <https://nethereum.com/>. — 2018.
- 11 Solidity programming language documentation. — <https://docs.soliditylang.org/en/v0.8.14/>.
- 12 Entity Framework Core ORM documentation. — <https://docs.microsoft.com/en-us/ef/>.